

# Deep Learning - III

RRC Summer School 2026

Recap

# Activation Functions

# Decision Boundaries

- Hidden layer output

$$z_i = w_{0,i}^{(1)} + \sum_{j=1}^m x_j w_{j,i}^{(1)}$$

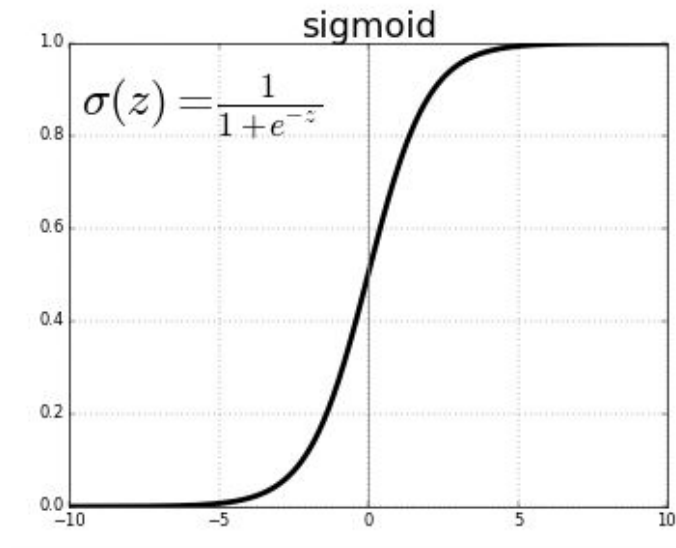
- Final output without activation

$$\hat{y}_i = w_{0,i}^{(2)} + \sum_{j=1}^{d_1} z_j w_{j,i}^{(2)}$$

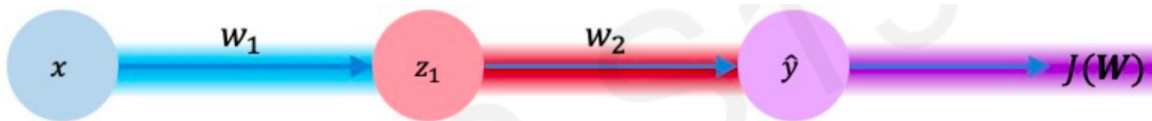
$$\hat{y}_i = w_{0,i}^{(2)} + \sum_{j=1}^{d_1} \left( w_{0,j}^{(1)} + \sum_{k=1}^m x_k w_{k,j}^{(1)} \right) w_{j,i}^{(2)}$$

$$\hat{y}_i = \left( w_{0,i}^{(2)} + \sum_{j=1}^{d_1} w_{0,j}^{(1)} \cdot w_{j,i}^{(2)} \right) + \sum_{k=1}^m x_k \left( \sum_{j=1}^{d_1} w_{k,j}^{(1)} \cdot w_{j,i}^{(2)} \right)$$

$$\hat{y}_i = w'_{0,i} + \sum_{k=1}^m x_k w'_{k,i}$$

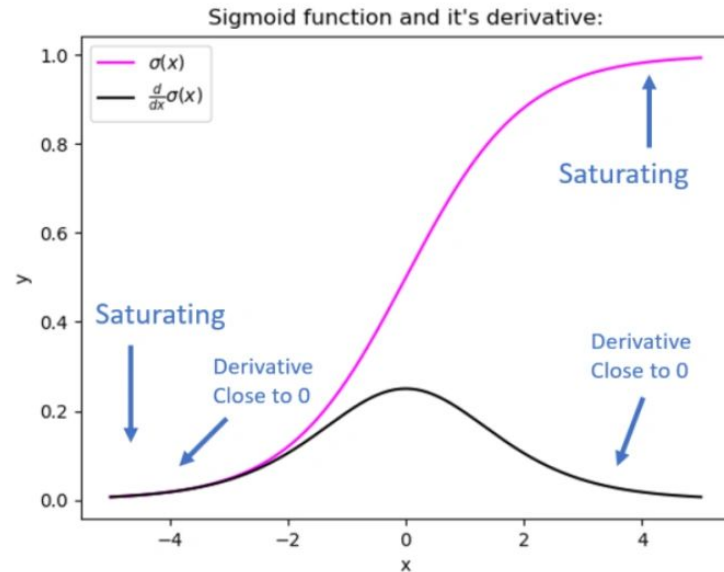


# Vanishing Gradients



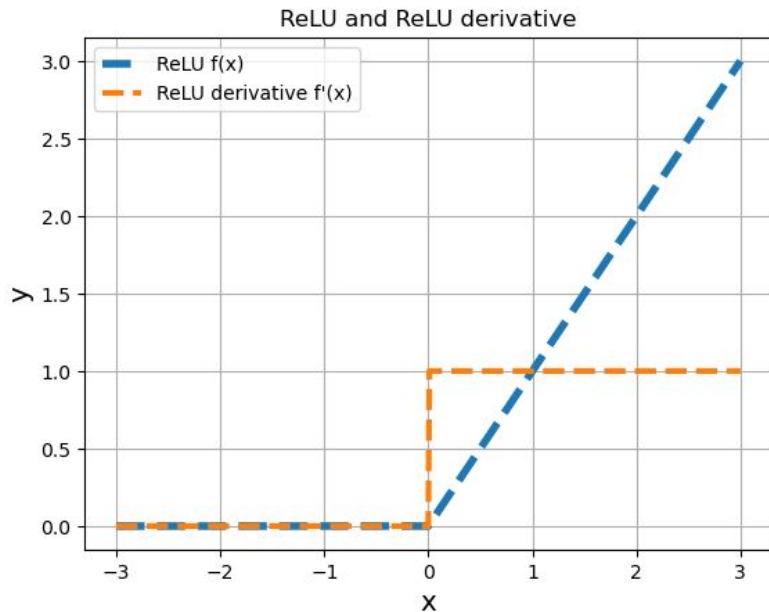
$$\frac{\partial J(W)}{\partial w_1} = \frac{\partial J(W)}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial z_1} * \frac{\partial z_1}{\partial w_1}$$

The equation is color-coded to match the diagram: the first term  $\frac{\partial J(W)}{\partial \hat{y}}$  is purple, the second term  $\frac{\partial \hat{y}}{\partial z_1}$  is red, and the third term  $\frac{\partial z_1}{\partial w_1}$  is blue.

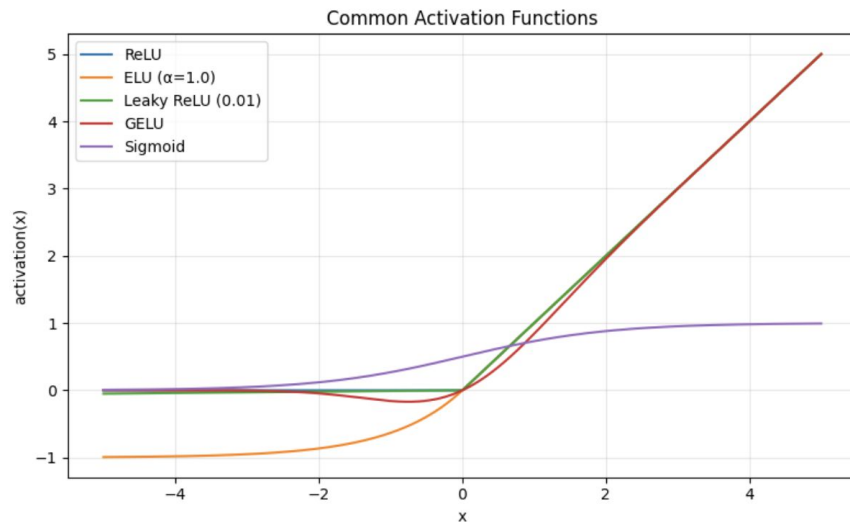


# ReLU

- ReLU:  $f(x) = \max(0, x)$
- Issue: Dead neurons / dying ReLU

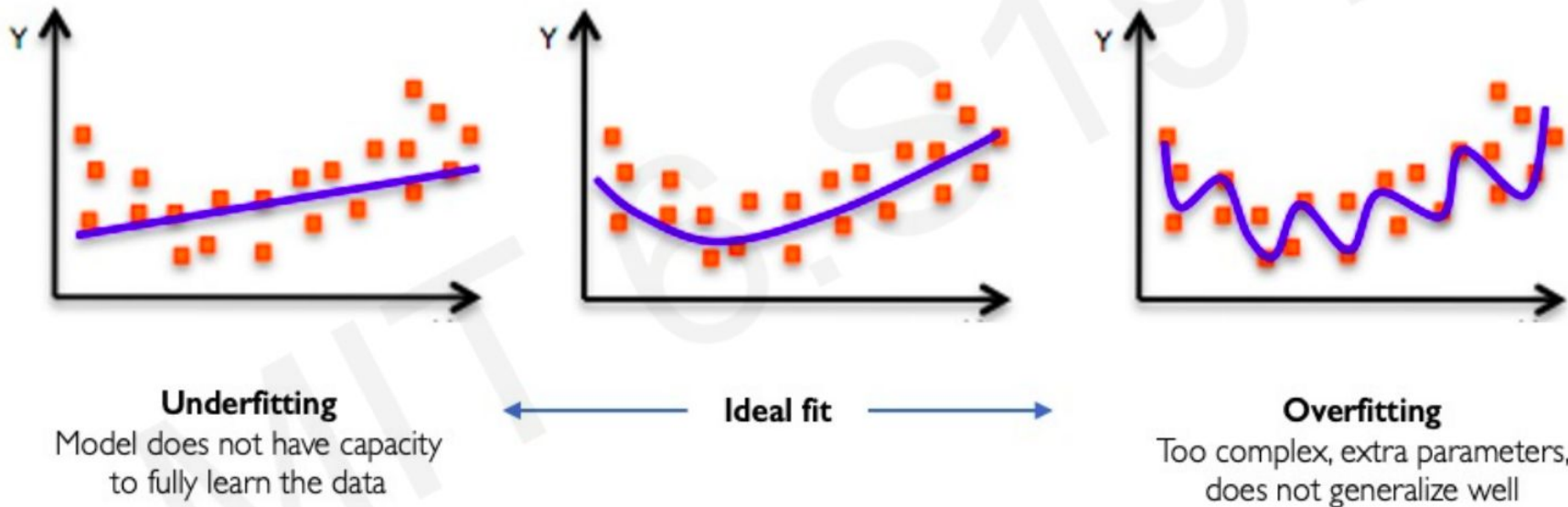


- Leaky ReLU
- Parametric ReLU
- ELU
- GELU
- SiLU



# Regularization

# The Problem of Overfitting





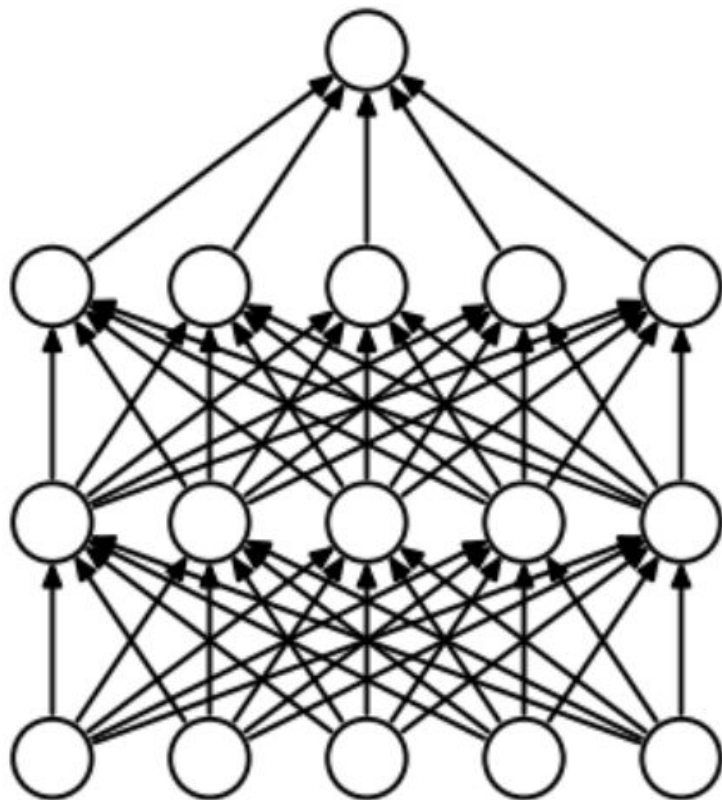
# Regularization

$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i) + \lambda R(W)$$

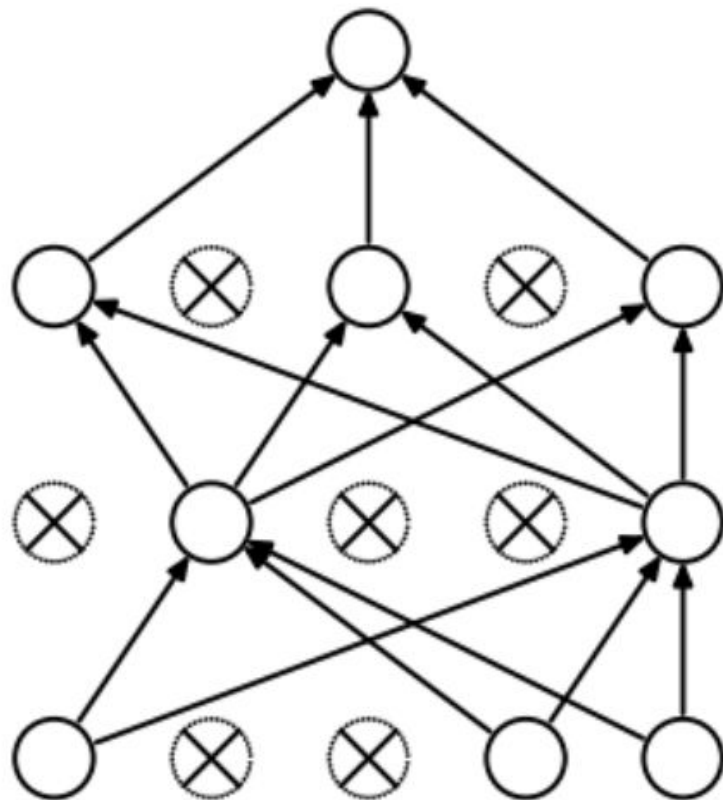
## Simple examples

- L2 regularization:  $R(W) = \sum_k \sum_l W_{k,l}^2$
- L1 regularization:  $R(W) = \sum_k \sum_l |W_{k,l}|$

# Regularization – Dropout



(a) Standard Neural Net



(b) After applying dropout.

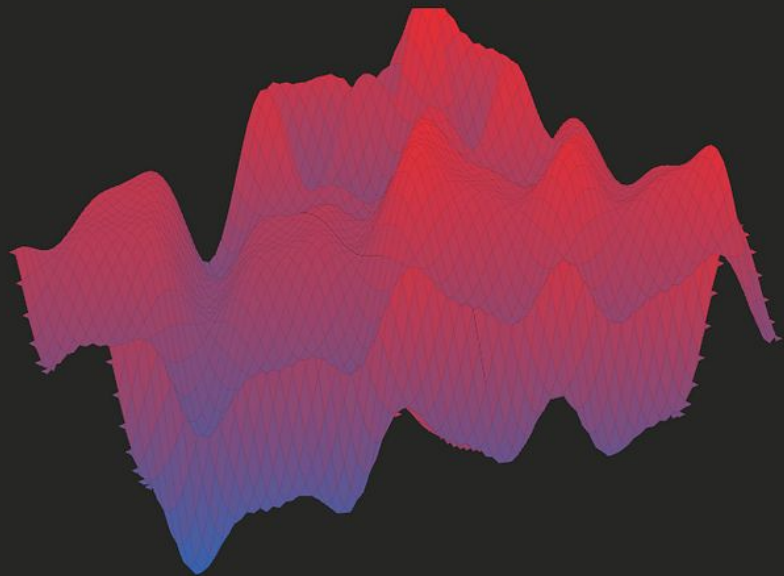
# Regularization – Early Stopping



# Normalization

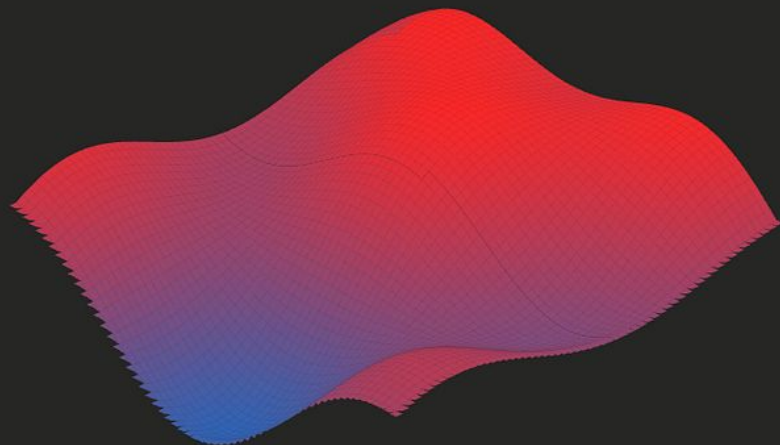
# Smoothing the Landscape

Without weight decay



Sharp peaks, narrow valleys

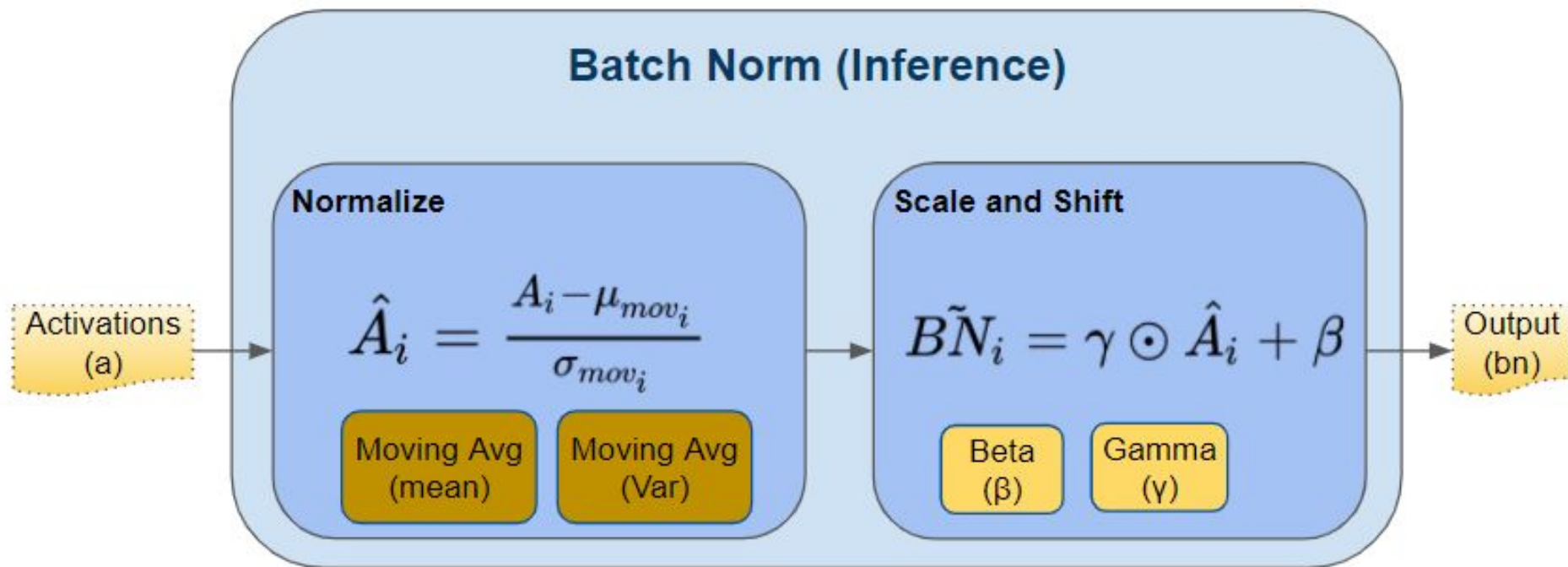
With weight decay



Smooth, wide valleys

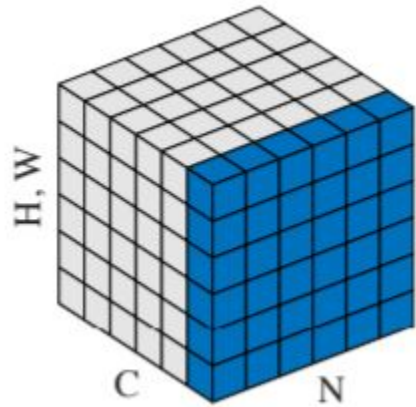
— Low loss    — High loss

# BatchNorm

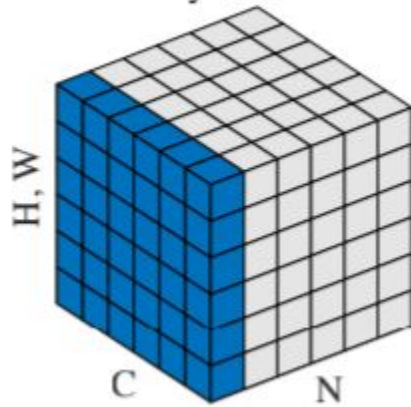


# Other Norms

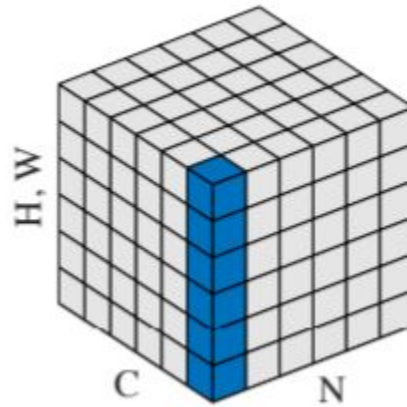
Batch Norm



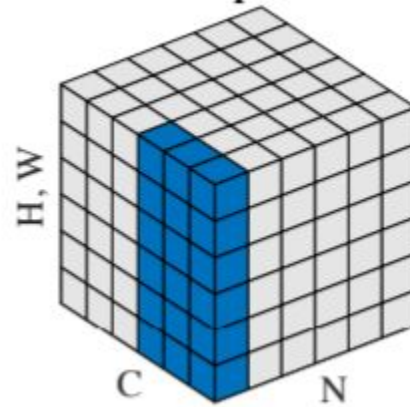
Layer Norm



Instance Norm



Group Norm



# Data Augmentation



# Augmentation

- Not just more data
- Rather about teaching the model what is important
- Improves generalization

# Augmentation



(a) Original



(b) Change Aspect Ratio



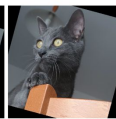
(c) Resizing



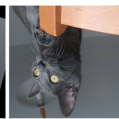
(d) Crop



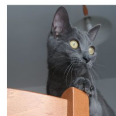
(e) Rotation + pad



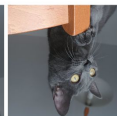
(f) Rotation



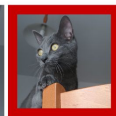
(g) Flip y



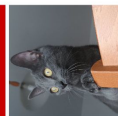
(h) Flip x



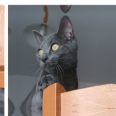
(i) Flip xy



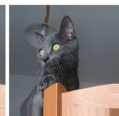
(j) Pad constant



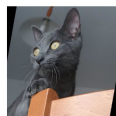
(k) Transpose



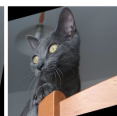
(l) Pad reflect



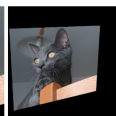
(m) Pad replicate



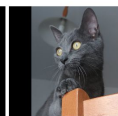
(n) Shear x



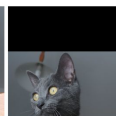
(o) Shear y



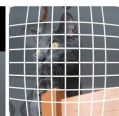
(p) Perspective



(q) Translation x



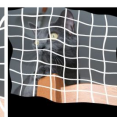
(r) Translation y



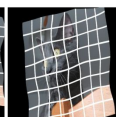
(s) Barrel



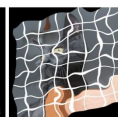
(t) Pincushion



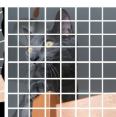
(u) Elastic



(v) Elastic, large  $\sigma$



(w) Elastic, small  $\sigma$



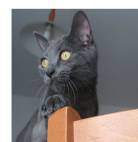
(x) Grid



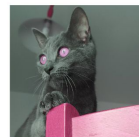
(y) Thin Plate Spline



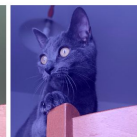
(z) Piecewise Affine



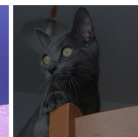
(a) Original



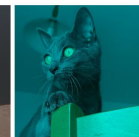
(b) Channel Shuffle



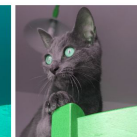
(c) Channel Shift



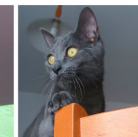
(d) Channel Multiply



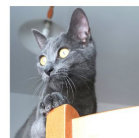
(e) Channel Dropout



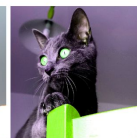
(f) Hue



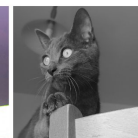
(g) Saturation



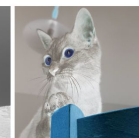
(h) Brightness



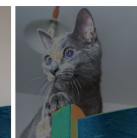
(i) Color Jitter



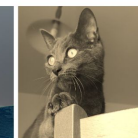
(j) Grayscale



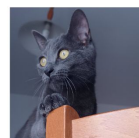
(k) Invert



(l) Solarization



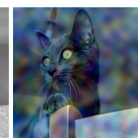
(m) Sepia



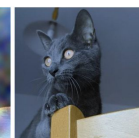
(n) FancyPCA



(o) Intensity Remap



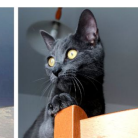
(p) FDA Simple Style Transfer\*



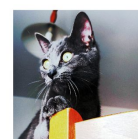
(q) Pixel Distribution Adaptation\*



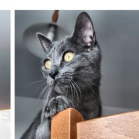
(r) Histogram Matching\*



(s) Contrast



(t) Hist Equalization



(u) CLAHE

# Augmentation



(a) Original



(b) AverageBlur

(c) MedianBlur

(d) GaussianBlur

(e) Generalized Normal Filter

(f) BilateralBlur

(g) JointBilateralBlur



(h) Defocus

(i) GlassBlur

(j) ZoomBlur

(k) MotionBlur

(l) UnsharpMask

(m) Emboss



(n) Dilation

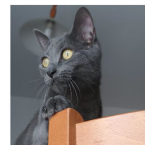
(o) Erosion

(p) Opening

(q) Closing

(r) Posterize

(s) Cartoon



(a) Original



(b) Salt

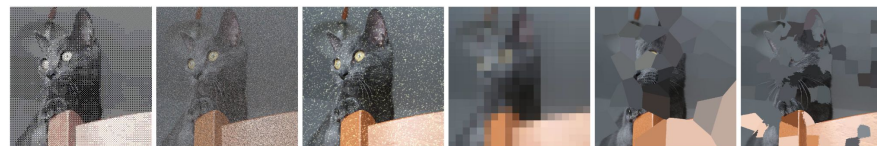
(c) Pepper

(d) Gaussian

(e) Laplace

(f) Poisson

(g) ISO



(h) Dithering

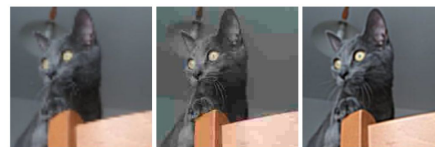
(i) ShufflePixels

(j) Spatter

(k) Pixelization

(l) Voronoi

(m) Superpixels



(n) Downscaling

(o) Compression

(p) Ringing

# Convolutional Neural Networks (CNNs)

# Images are Numbers



|     |     |     |     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 157 | 153 | 174 | 168 | 150 | 152 | 129 | 151 | 172 | 161 | 155 | 156 |
| 155 | 182 | 163 | 74  | 75  | 62  | 33  | 17  | 110 | 210 | 180 | 154 |
| 180 | 180 | 50  | 14  | 34  | 6   | 10  | 33  | 48  | 106 | 159 | 181 |
| 206 | 109 | 5   | 124 | 131 | 111 | 120 | 204 | 166 | 15  | 56  | 180 |
| 194 | 68  | 137 | 251 | 237 | 239 | 239 | 228 | 227 | 87  | 71  | 201 |
| 172 | 105 | 207 | 233 | 233 | 214 | 220 | 239 | 228 | 98  | 74  | 206 |
| 188 | 88  | 179 | 209 | 185 | 215 | 211 | 158 | 139 | 75  | 20  | 169 |
| 189 | 97  | 165 | 84  | 10  | 168 | 134 | 11  | 31  | 62  | 22  | 148 |
| 199 | 168 | 191 | 193 | 158 | 227 | 178 | 143 | 182 | 106 | 36  | 190 |
| 205 | 174 | 155 | 252 | 236 | 231 | 149 | 178 | 228 | 43  | 95  | 234 |
| 190 | 216 | 116 | 149 | 236 | 187 | 86  | 150 | 79  | 38  | 218 | 241 |
| 190 | 224 | 147 | 108 | 227 | 210 | 127 | 102 | 36  | 101 | 255 | 224 |
| 190 | 214 | 173 | 66  | 103 | 143 | 96  | 50  | 2   | 109 | 249 | 215 |
| 187 | 196 | 235 | 75  | 1   | 81  | 47  | 0   | 6   | 217 | 255 | 211 |
| 183 | 202 | 237 | 145 | 0   | 0   | 12  | 108 | 200 | 138 | 243 | 236 |
| 195 | 206 | 123 | 207 | 177 | 121 | 123 | 200 | 175 | 13  | 96  | 218 |

What the computer sees

|     |     |     |     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 157 | 153 | 174 | 168 | 150 | 152 | 129 | 151 | 172 | 161 | 155 | 156 |
| 155 | 182 | 163 | 74  | 75  | 62  | 33  | 17  | 110 | 210 | 180 | 154 |
| 180 | 180 | 50  | 14  | 34  | 6   | 10  | 33  | 48  | 106 | 159 | 181 |
| 206 | 109 | 5   | 124 | 131 | 111 | 120 | 204 | 166 | 15  | 56  | 180 |
| 194 | 68  | 137 | 251 | 237 | 239 | 239 | 228 | 227 | 87  | 71  | 201 |
| 172 | 105 | 207 | 233 | 233 | 214 | 220 | 239 | 228 | 98  | 74  | 206 |
| 188 | 88  | 179 | 209 | 185 | 215 | 211 | 158 | 139 | 75  | 20  | 169 |
| 189 | 97  | 165 | 84  | 10  | 168 | 134 | 11  | 31  | 62  | 22  | 148 |
| 199 | 168 | 191 | 193 | 158 | 227 | 178 | 143 | 182 | 106 | 36  | 190 |
| 205 | 174 | 155 | 252 | 236 | 231 | 149 | 178 | 228 | 43  | 95  | 234 |
| 190 | 216 | 116 | 149 | 236 | 187 | 86  | 150 | 79  | 38  | 218 | 241 |
| 190 | 224 | 147 | 108 | 227 | 210 | 127 | 102 | 36  | 101 | 255 | 224 |
| 190 | 214 | 173 | 66  | 103 | 143 | 96  | 50  | 2   | 109 | 249 | 215 |
| 187 | 196 | 235 | 75  | 1   | 81  | 47  | 0   | 6   | 217 | 255 | 211 |
| 183 | 202 | 237 | 145 | 0   | 0   | 12  | 108 | 200 | 138 | 243 | 236 |
| 195 | 206 | 123 | 207 | 177 | 121 | 123 | 200 | 175 | 13  | 96  | 218 |

An image is just a matrix of numbers  $[0,255]$ !  
i.e.,  $1080 \times 1080 \times 3$  for an RGB image

# Manual Feature Extraction

Domain  
knowledge

Define features

Detect features  
to classify

Viewpoint variation



Scale variation



Deformation



Occlusion



Illumination conditions



Background clutter



Intra-class variation

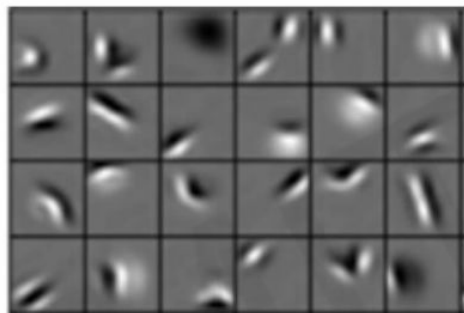




# Learning Feature Representations

Can we learn a **hierarchy of features** directly from the data instead of hand engineering?

Low level features



Edges, dark spots

Mid level features



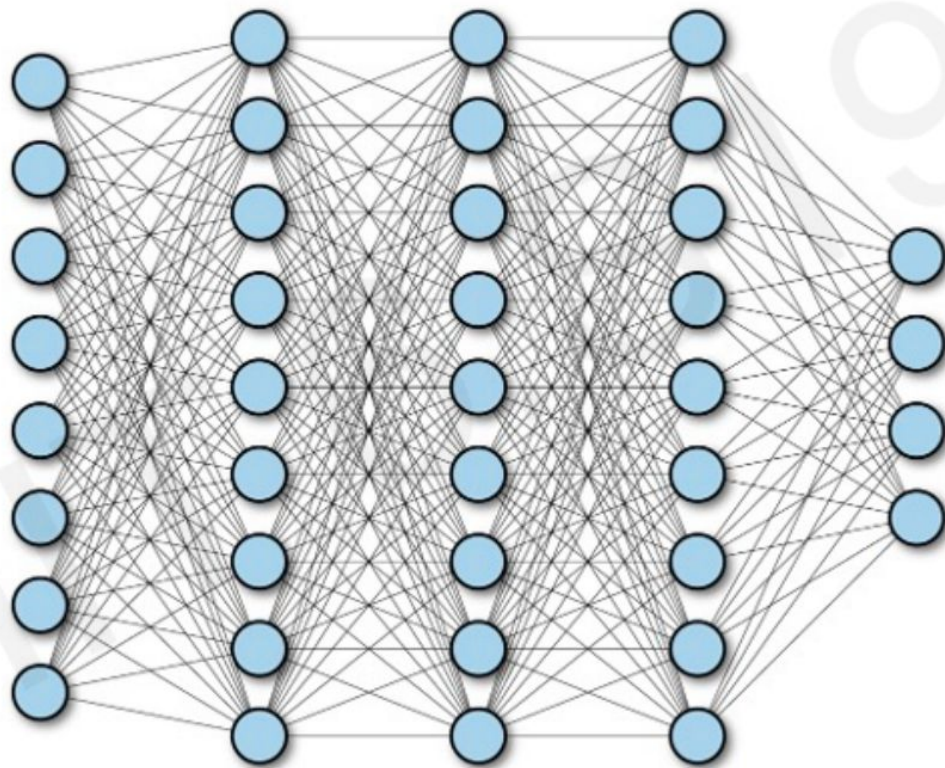
Eyes, ears, nose

High level features



Facial structure

# Fully Connected Neural Network

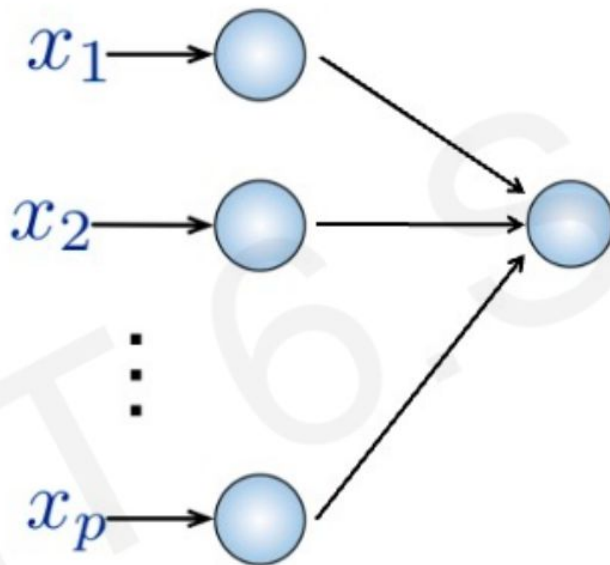




# Fully Connected Neural Network

## Input:

- 2D image
- Vector of pixel values



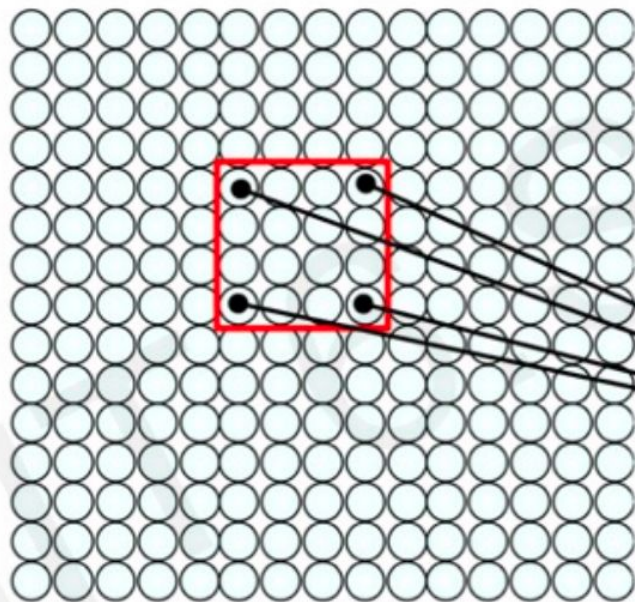
## Fully Connected:

- Connect neuron in hidden layer to all neurons in input layer
- No spatial information!
- And many, many parameters!

How can we use **spatial structure** in the input to inform the architecture of the network?

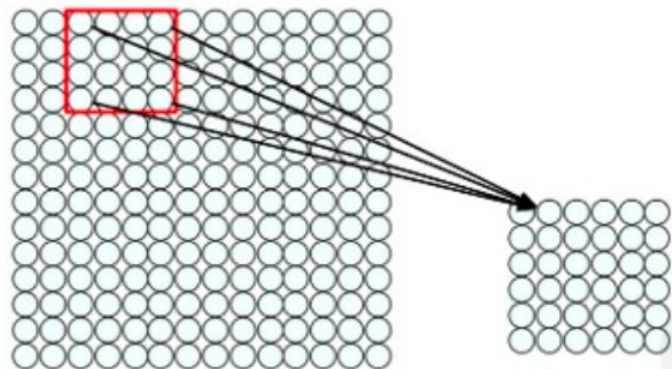
# Using Spatial Structure

**Input:** 2D image.  
Array of pixel values



**Idea:** connect patches of input  
to neurons in hidden layer.  
Neuron connected to region of  
input. Only "sees" these values.

# Feature Extraction with Convolution



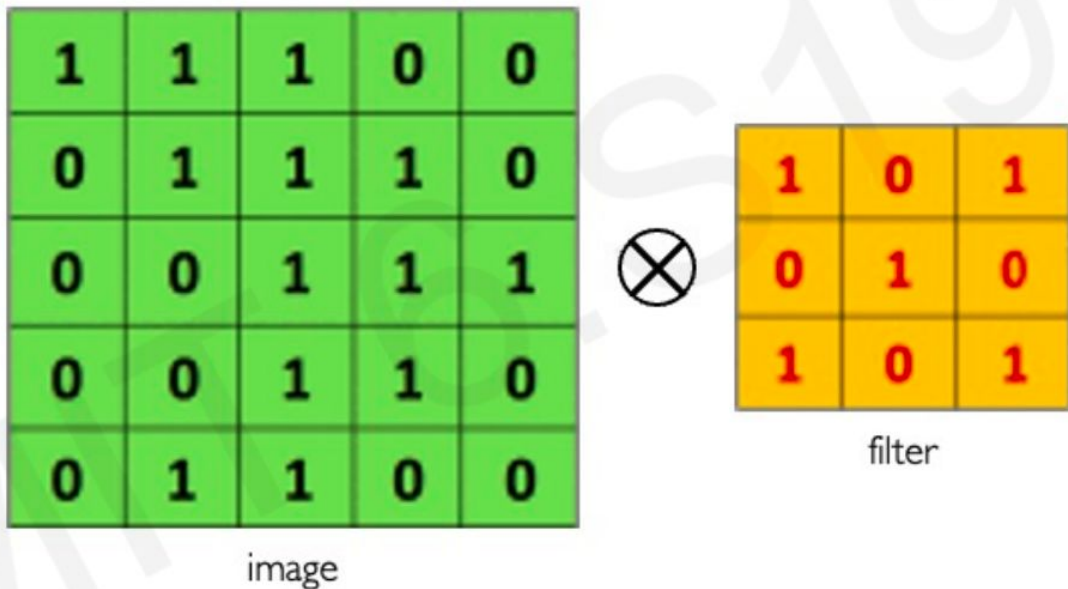
- Filter of size 4x4 : 16 different weights
- Apply this same filter to 4x4 patches in input
- Shift by 2 pixels for next patch

This “patchy” operation is **convolution**

- 1) Apply a set of weights – a filter – to extract **local features**
- 2) Use **multiple filters** to extract different features
- 3) **Spatially share** parameters of each filter

# The Convolution Operation

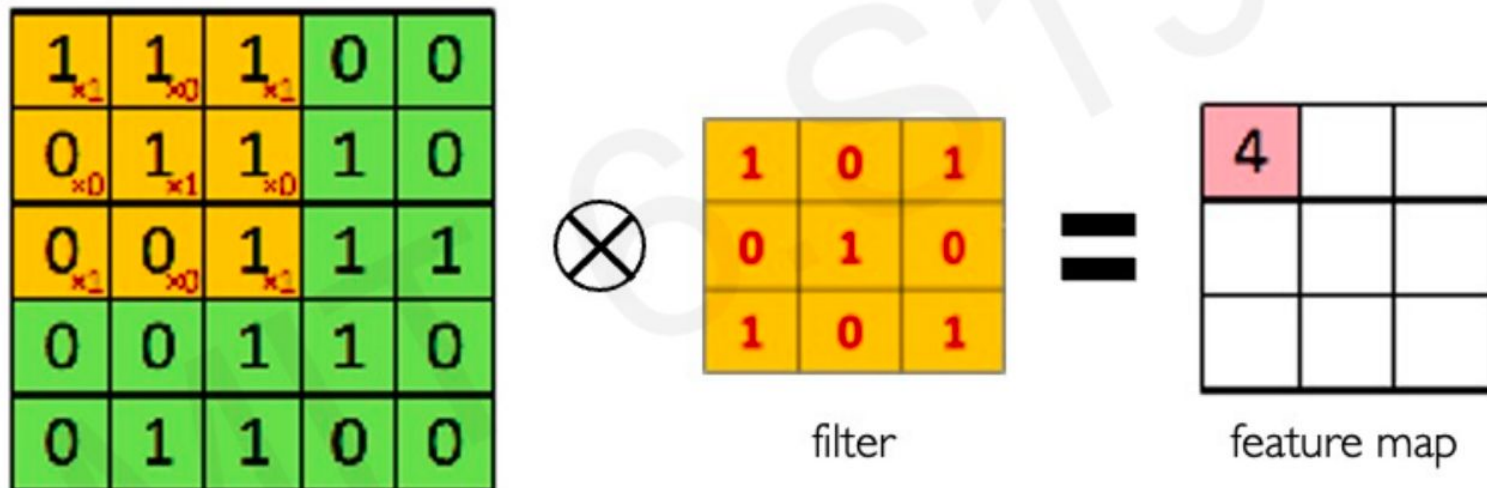
Suppose we want to compute the convolution of a 5x5 image and a 3x3 filter:



We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs...

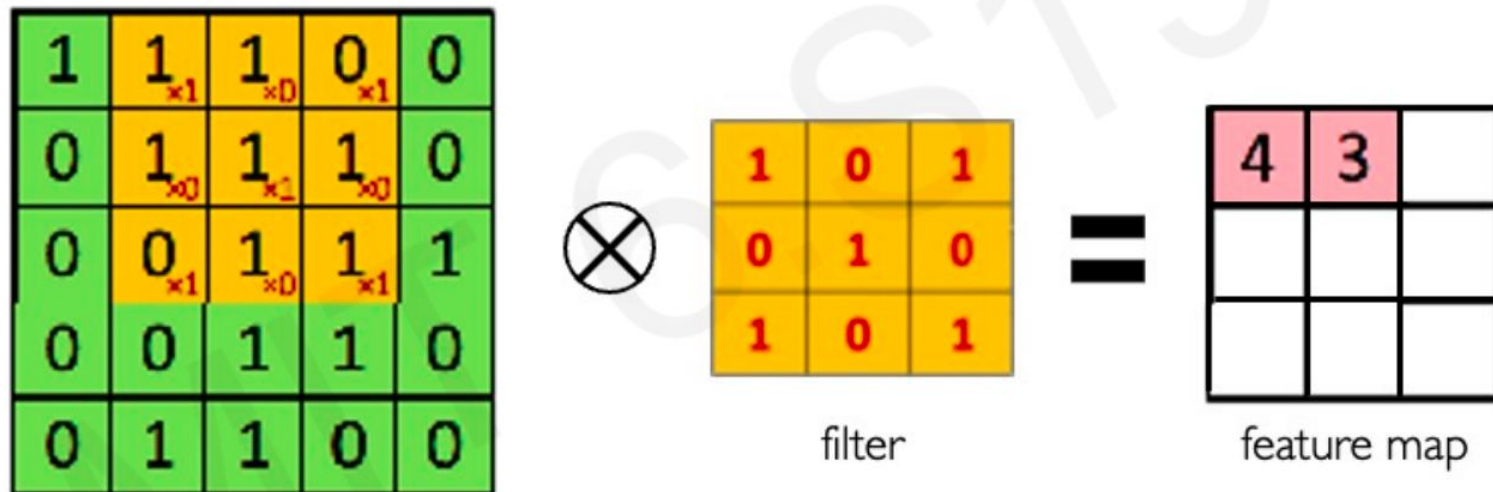
# The Convolution Operation

We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs:



# The Convolution Operation

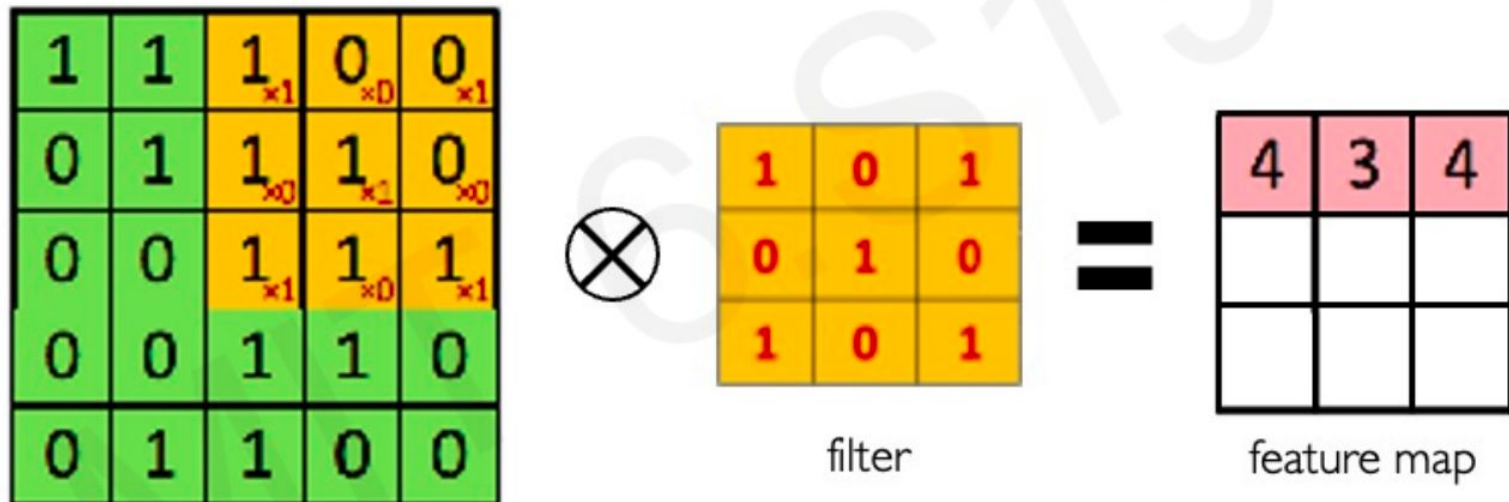
We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs:





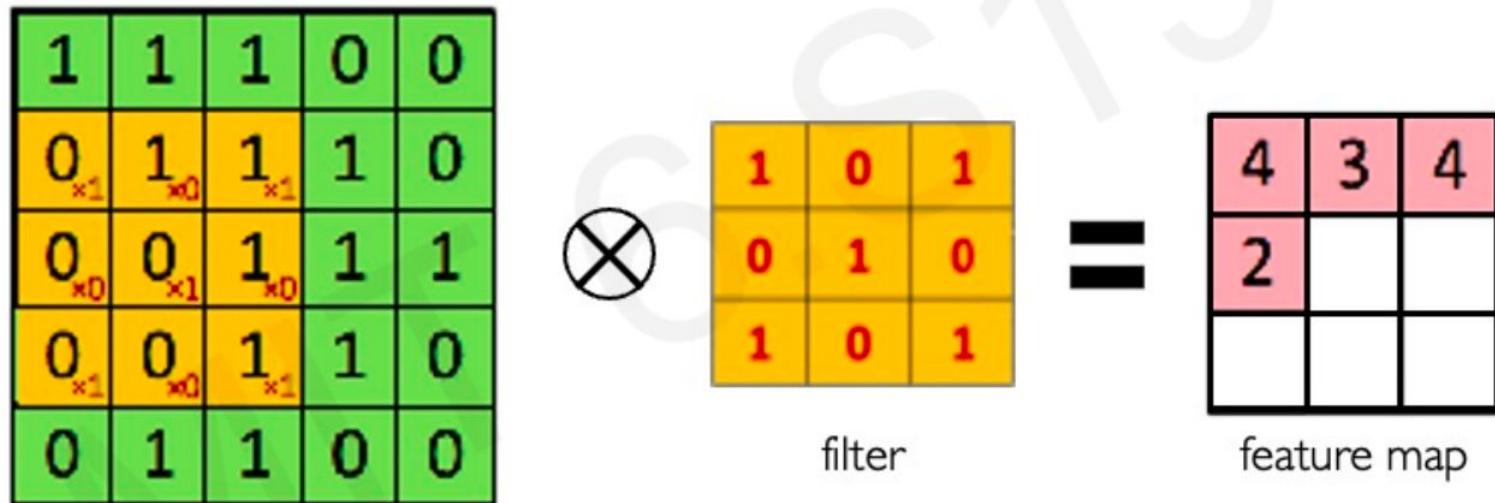
# The Convolution Operation

We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs:



# The Convolution Operation

We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs:





# The Convolution Operation

We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs:

|   |                 |                 |                 |   |
|---|-----------------|-----------------|-----------------|---|
| 1 | 1               | 1               | 0               | 0 |
| 0 | 1 <sub>x1</sub> | 1 <sub>x0</sub> | 1 <sub>x1</sub> | 0 |
| 0 | 0 <sub>x0</sub> | 1 <sub>x1</sub> | 1 <sub>x0</sub> | 1 |
| 0 | 0 <sub>x1</sub> | 1 <sub>x0</sub> | 1 <sub>x1</sub> | 0 |
| 0 | 1               | 1               | 0               | 0 |



|   |   |   |
|---|---|---|
| 1 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 1 |

filter

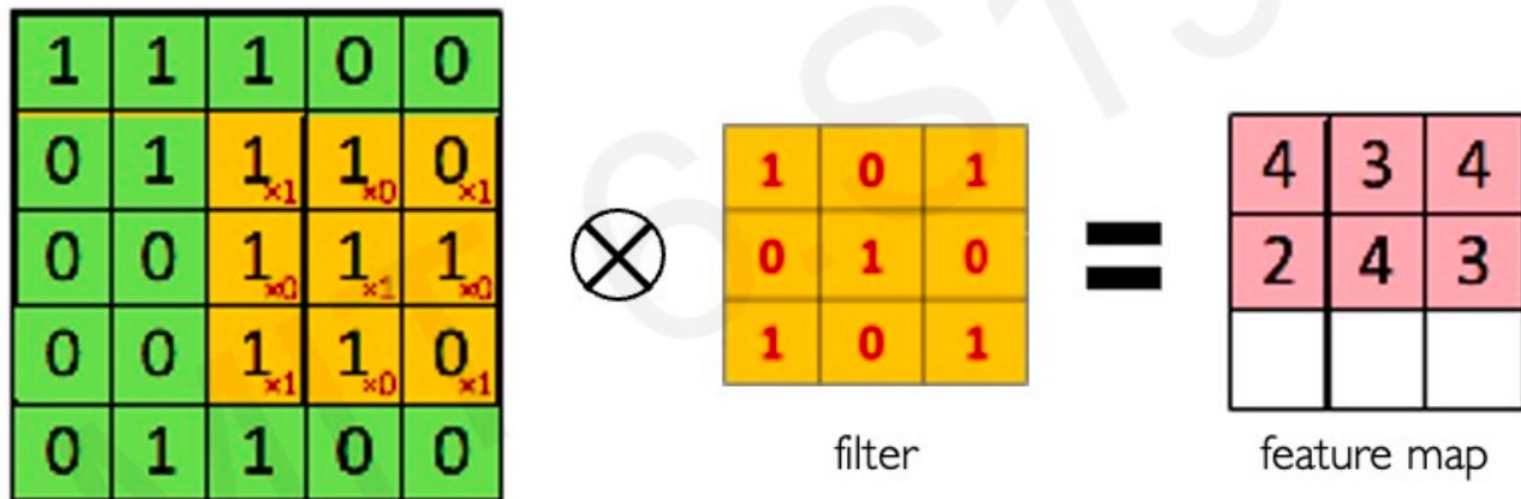


|   |   |   |
|---|---|---|
| 4 | 3 | 4 |
| 2 | 4 |   |
|   |   |   |

feature map

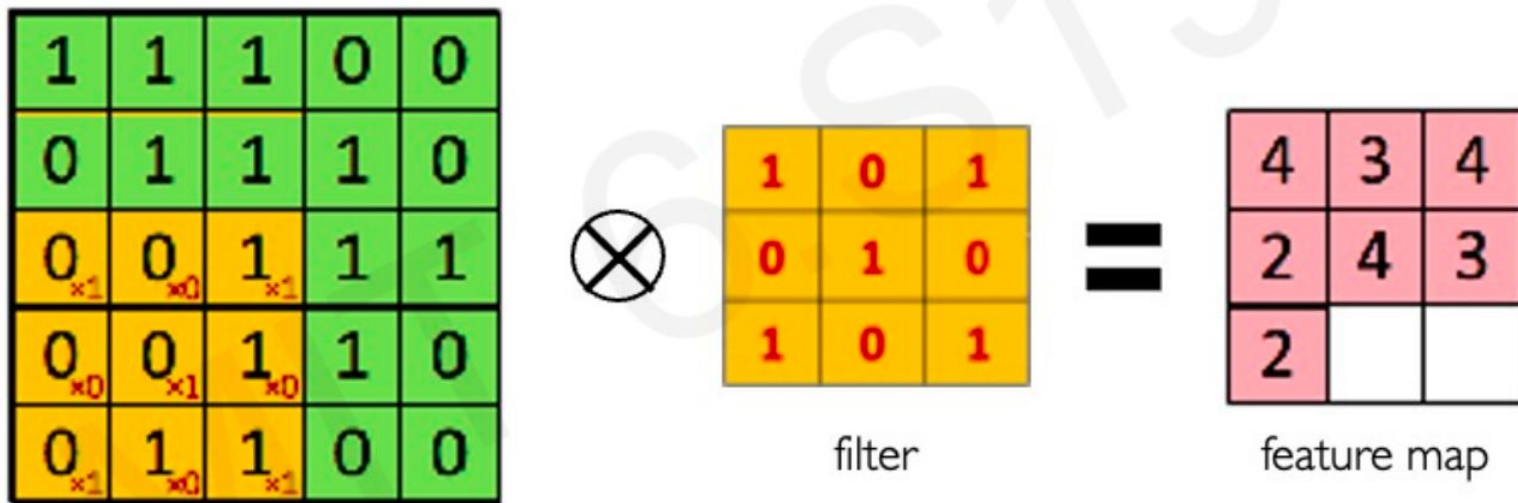
# The Convolution Operation

We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs:



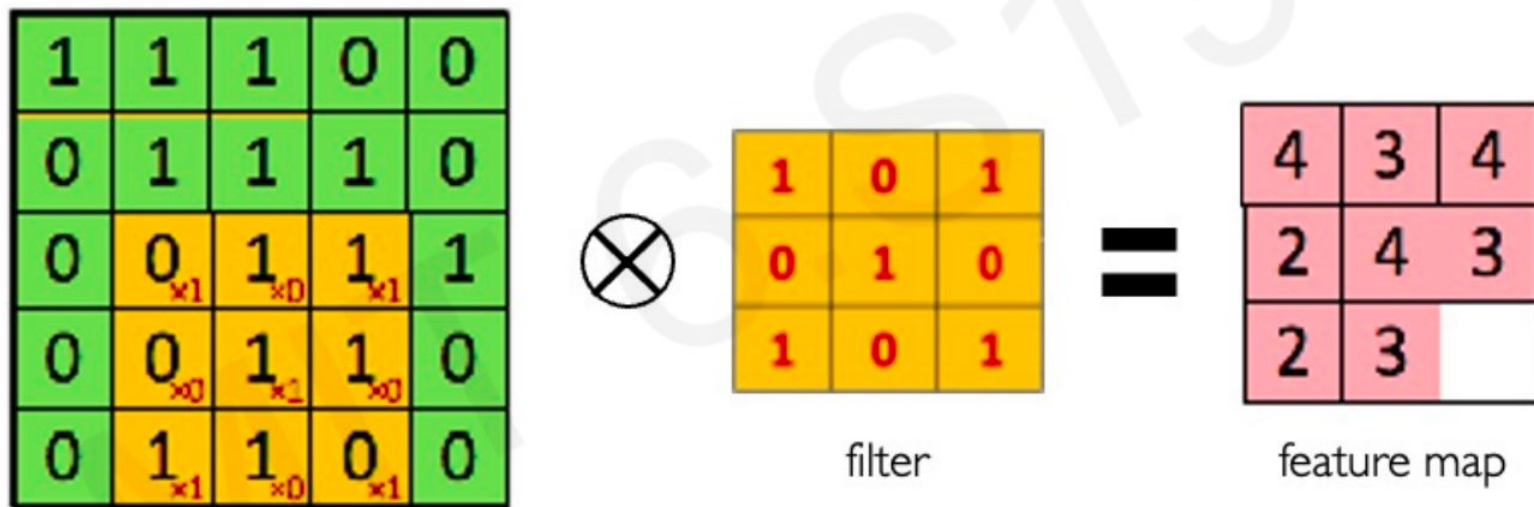
# The Convolution Operation

We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs:



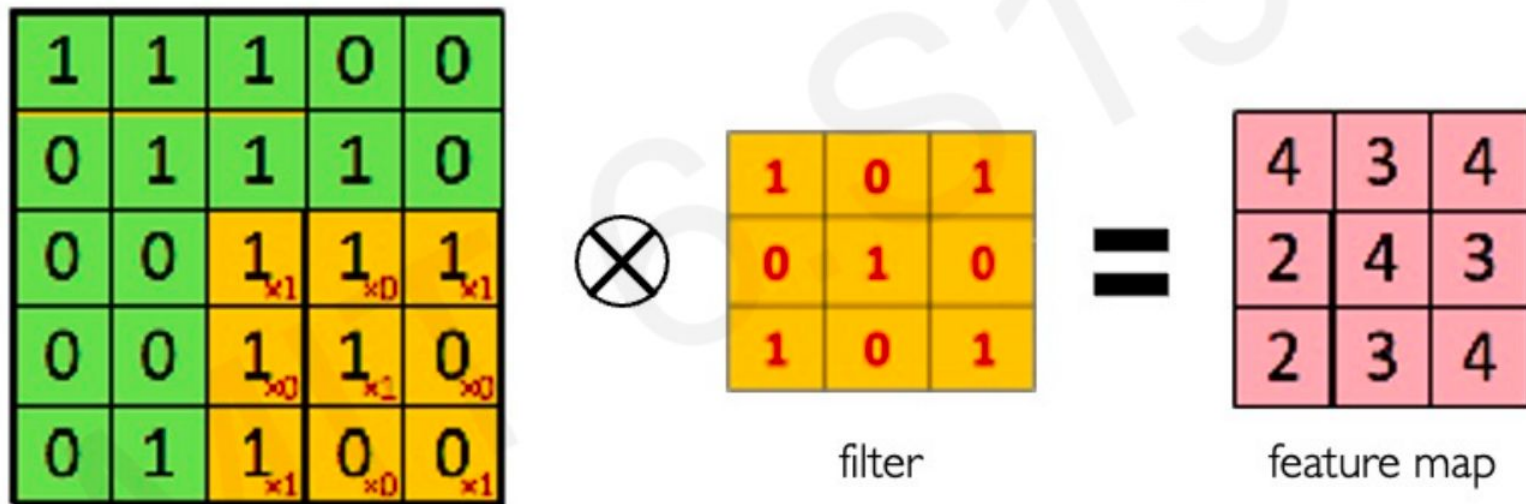
# The Convolution Operation

We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs:

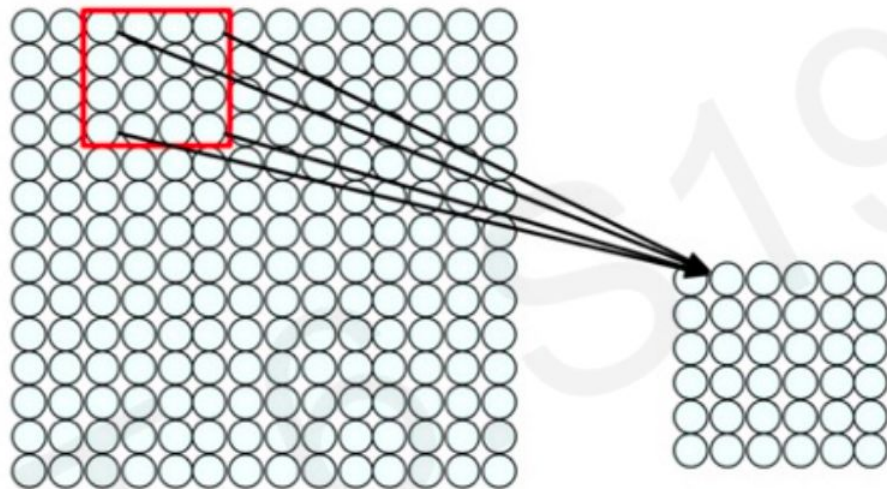


# The Convolution Operation

We slide the 3x3 filter over the input image, element-wise multiply, and add the outputs:



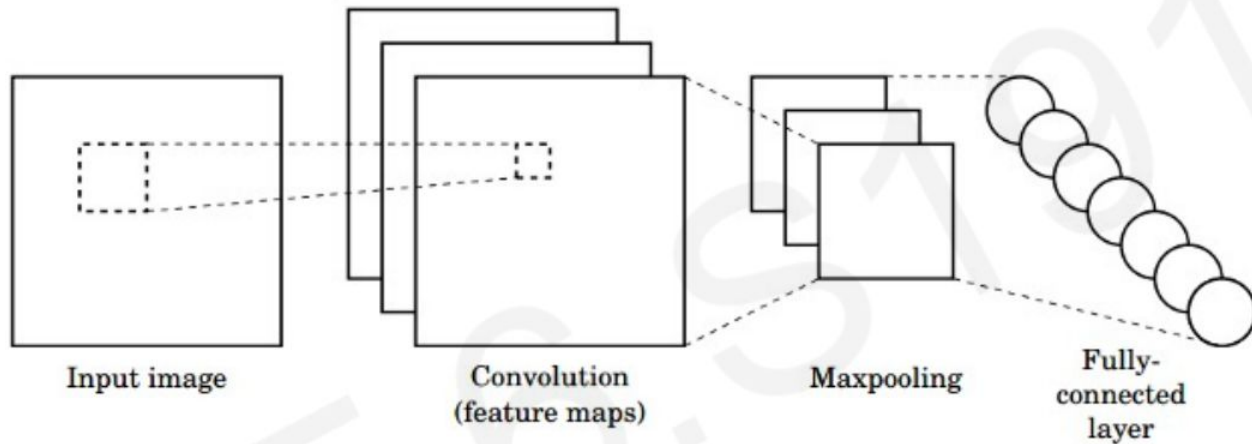
# Feature Extraction with Convolution



- 1) Apply a set of weights – a filter – to extract **local features**
- 2) Use **multiple filters** to extract different features
- 3) **Spatially share** parameters of each filter



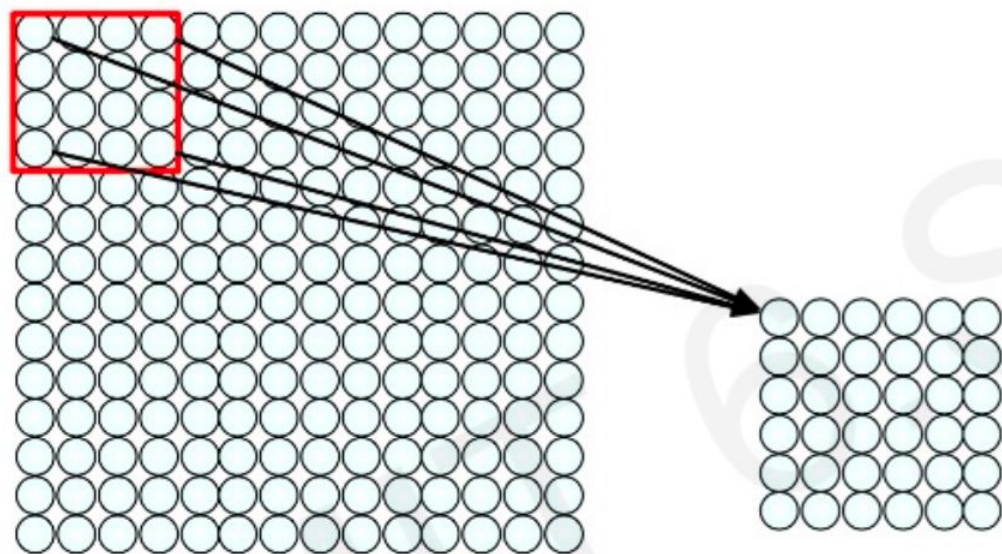
# CNNs for Classification



1. **Convolution**: Apply filters to generate feature maps.
2. **Non-linearity**: Often ReLU.
3. **Pooling**: Downsampling operation on each feature map.

**Train model with image data.**  
**Learn weights of filters in convolutional layers.**

# Convolutional Layers: Local Connectivity



**For a neuron in hidden layer:**

- Take inputs from patch
- Compute weighted sum
- Apply bias

4x4 filter: matrix  
of weights  $w_{ij}$

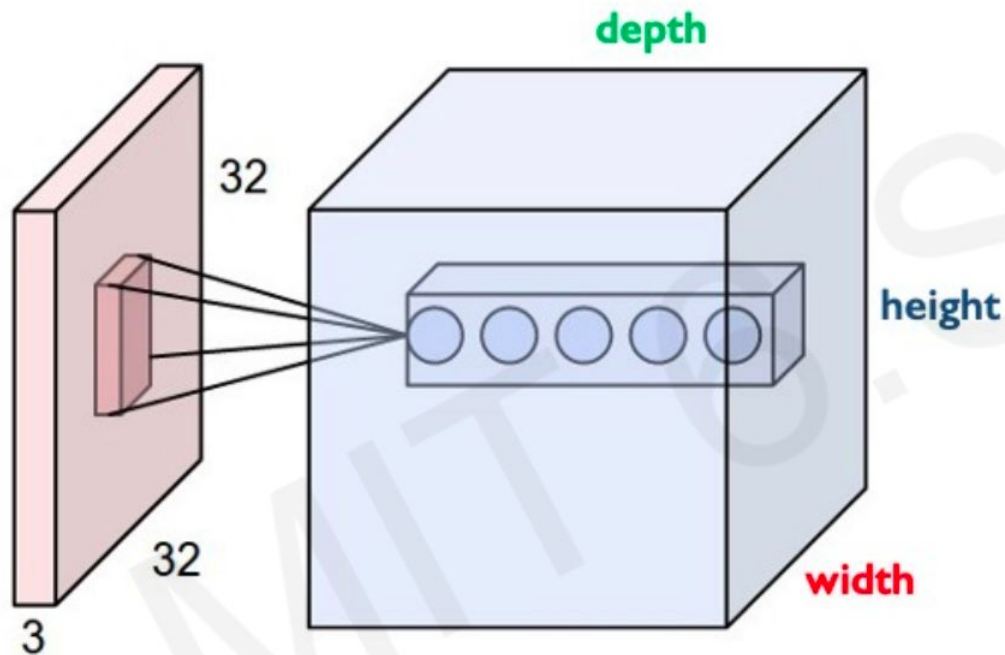
$$\sum_{i=1}^4 \sum_{j=1}^4 w_{ij} x_{i+p,j+q} + b$$

for neuron (p,q) in hidden layer

- 1) applying a window of weights
- 2) computing linear combinations
- 3) activating with non-linear function



# CNNs: Spatial Arrangement of Output Volume



## Layer Dimensions:

$h \times w \times d$

where  $h$  and  $w$  are spatial dimensions  
 $d$  (depth) = number of filters

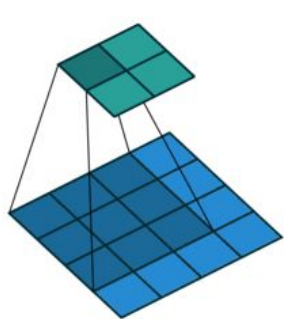
## Stride:

Filter step size

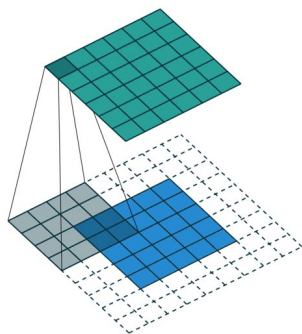
## Receptive Field:

Locations in input image that  
a node is path connected to

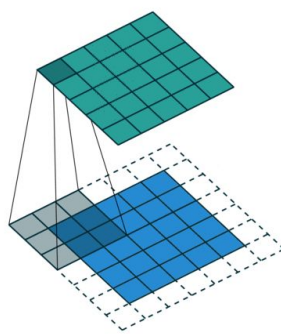
# Padding and Stride



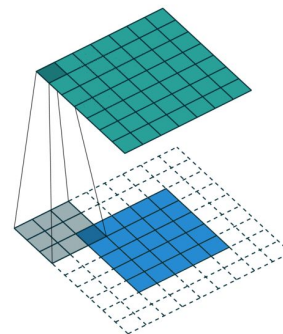
No padding,  
no strides



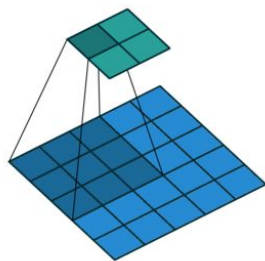
Arbitrary padding,  
no strides



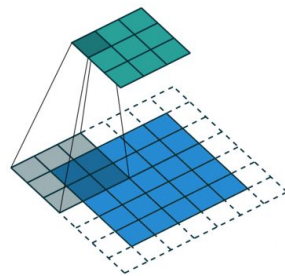
Half padding,  
no strides



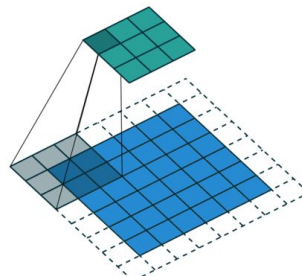
Full padding,  
no strides



No padding,  
strides



Padding,  
strides



Padding,  
Strides (odd)

# Output feature dimension

- Consider that we have an input feature map with dimensions  $W \times W \times D$
- We also have  $D'$  filters/kernels each with dimensions  $F \times F$ , padding  $P$  and stride  $S$
- What will be the dimension of the output feature map  $W_{out}$ ?

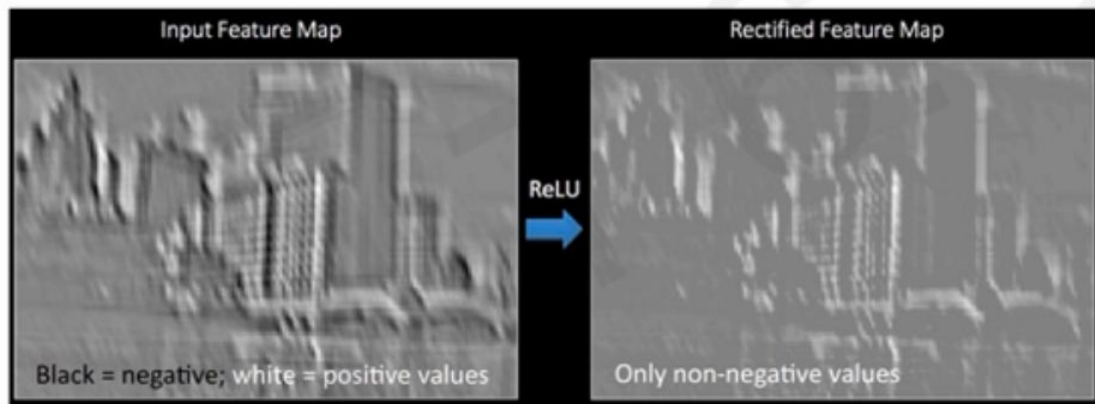
$$W_{out} = \frac{W - F + 2P}{S} + 1$$

<https://towardsdatascience.com/convolutional-neural-networks-explained-9cc5188c4939>

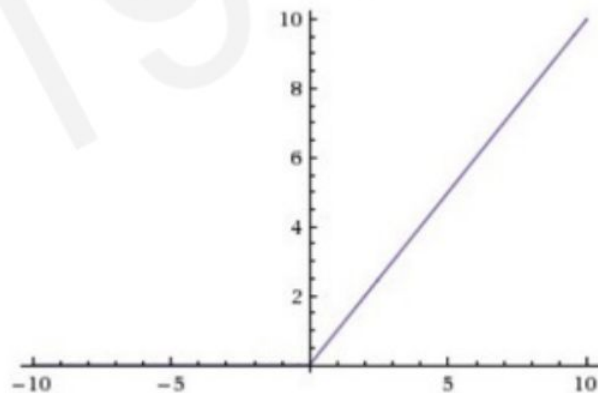
- Additionally, since there are  $D'$  filters that are being applied, the final output dimension is  $W_{out} \times W_{out} \times D'$

# Introducing Non-Linearity

- Apply after every convolution operation (i.e., after convolutional layers)
- ReLU: pixel-by-pixel operation that replaces all negative values by zero. **Non-linear operation!**



## Rectified Linear Unit (ReLU)



$$g(z) = \max(0, z)$$

# Pooling

x ↑

|   |   |   |   |
|---|---|---|---|
| 1 | 1 | 2 | 4 |
| 5 | 6 | 7 | 8 |
| 3 | 2 | 1 | 0 |
| 1 | 2 | 3 | 4 |

→ y

max pool with 2x2 filters  
and stride 2



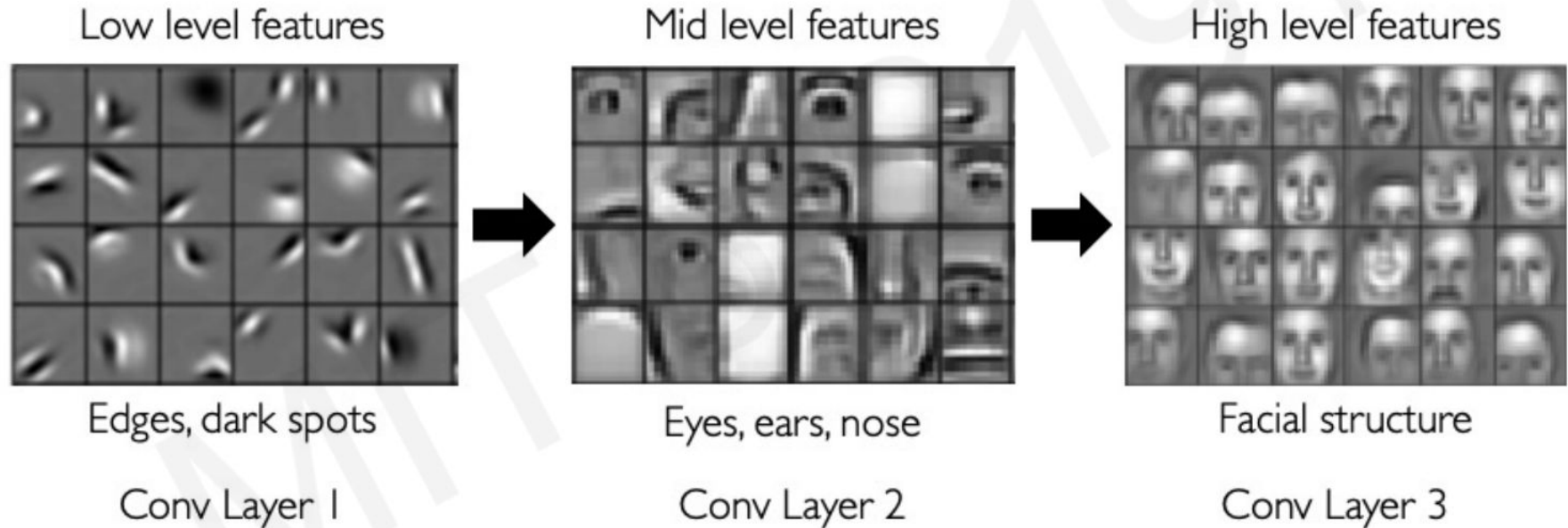
|   |   |
|---|---|
| 6 | 8 |
| 3 | 4 |

- 1) Reduced dimensionality
- 2) Spatial invariance

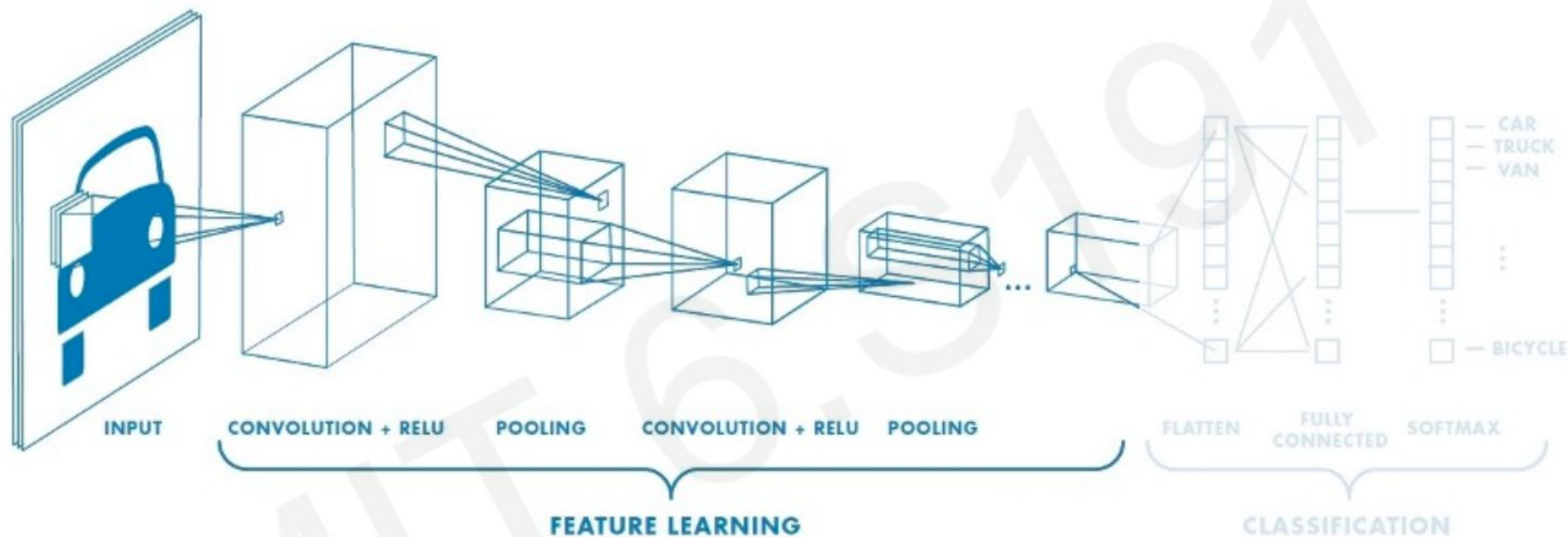
How else can we downsample and preserve spatial invariance?

Average Pooling!

# Representation Learning in Deep CNNs



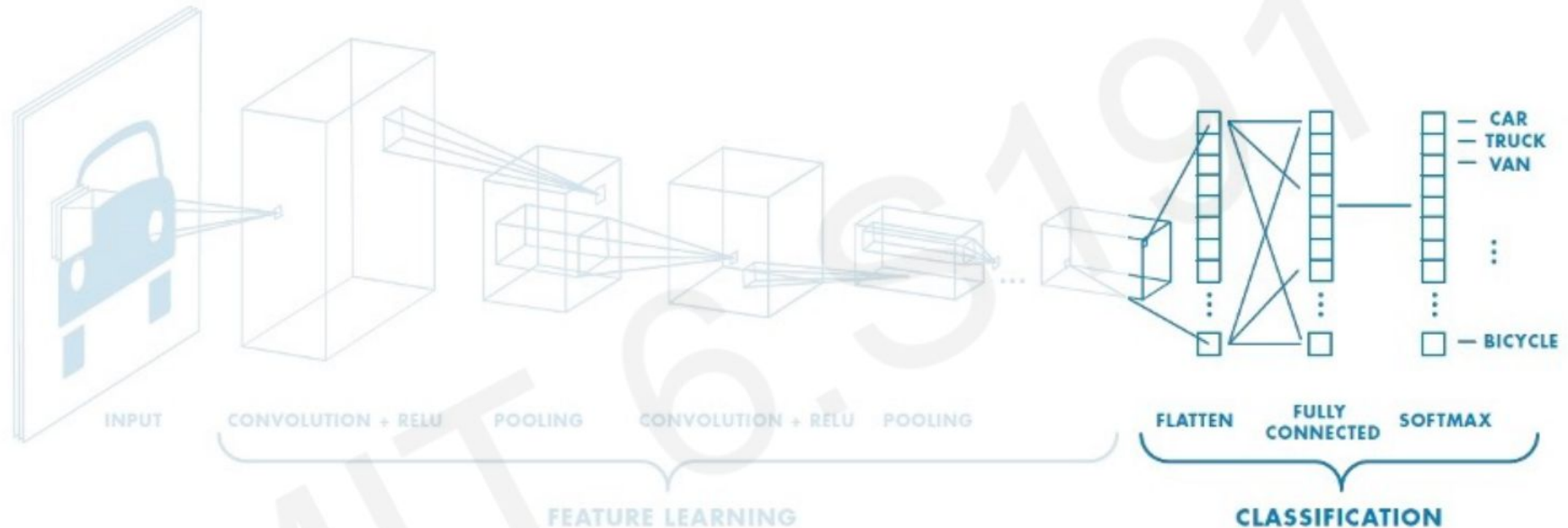
# CNNs for Classification: Feature Learning



1. Learn features in input image through **convolution**
2. Introduce **non-linearity** through activation function (real-world data is non-linear!)
3. Reduce dimensionality and preserve spatial invariance with **pooling**



# CNNs for Classification: Class Probabilities

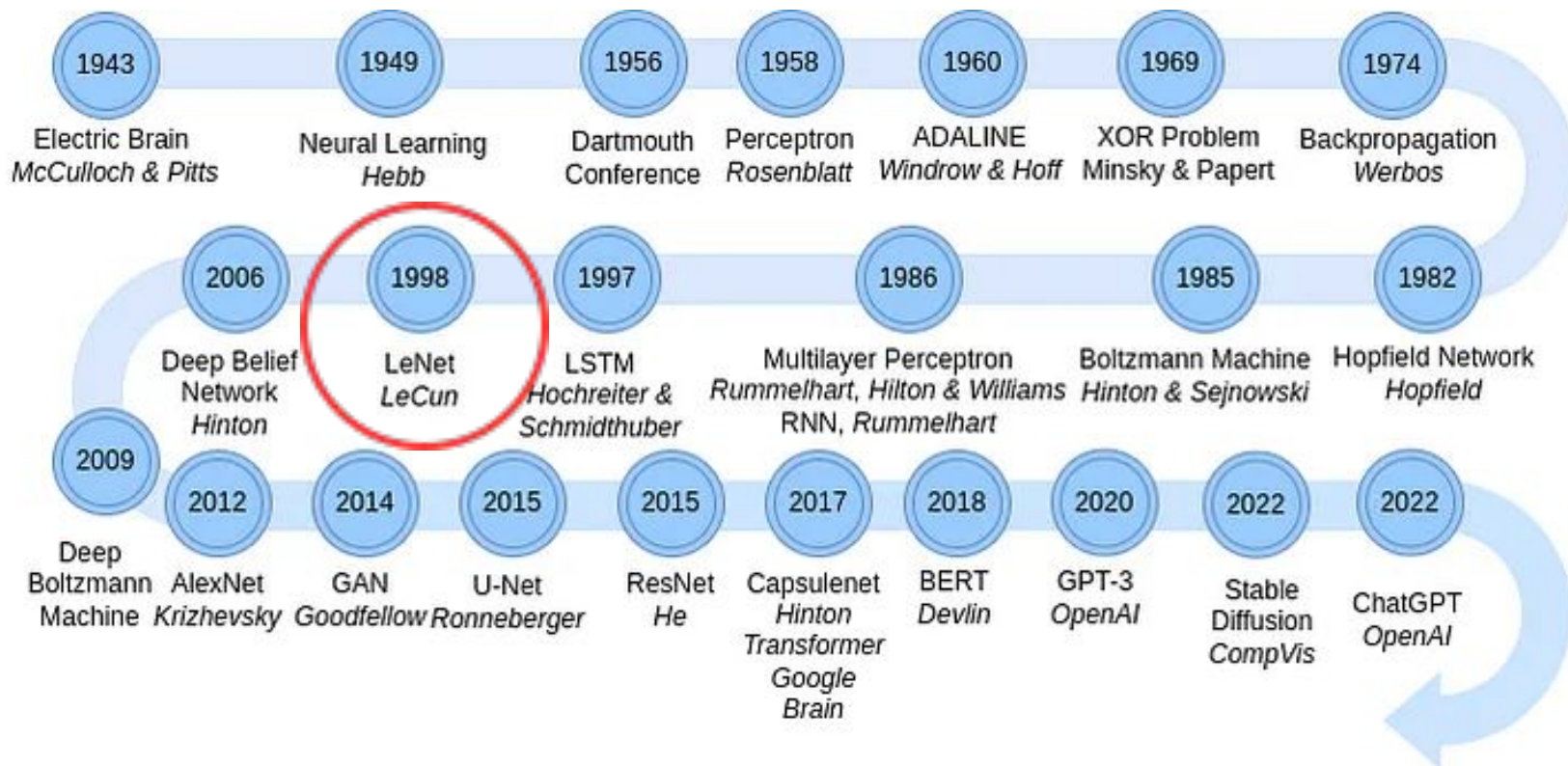


- CONV and POOL layers output high-level features of input
- Fully connected layer uses these features for classifying input image
- Express output as **probability** of image belonging to a particular class

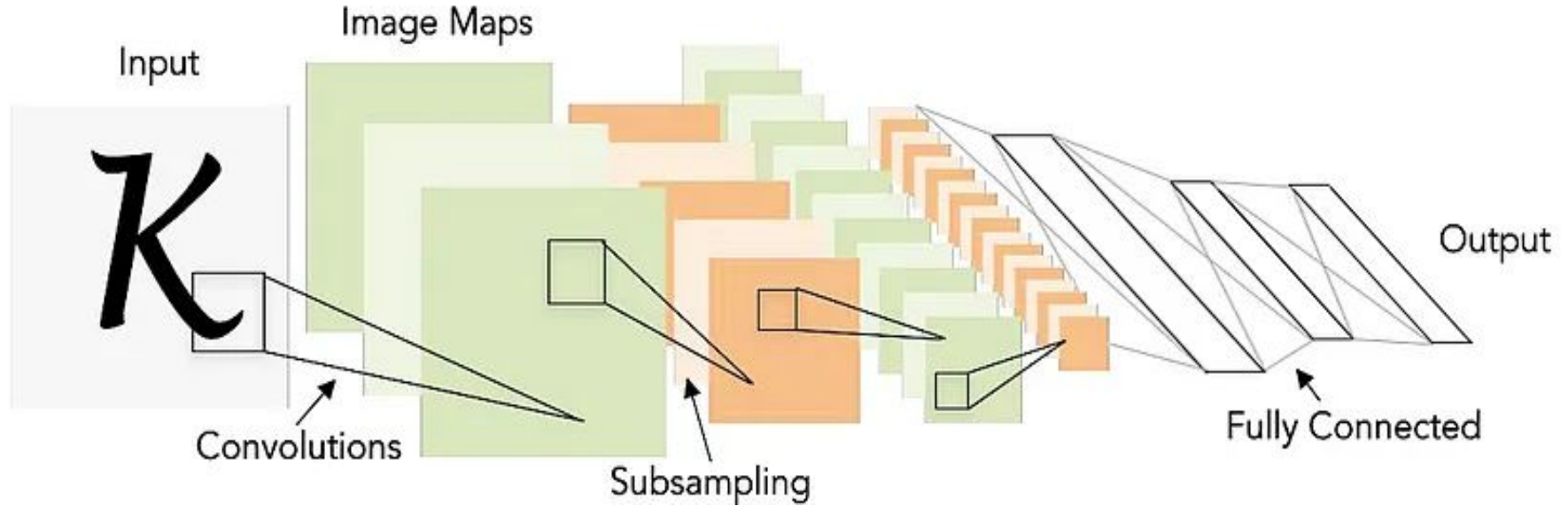
$$\text{softmax}(y_i) = \frac{e^{y_i}}{\sum_j e^{y_j}}$$

Brief history of various CNN architectures

# LeNet-5 (1989-1998)

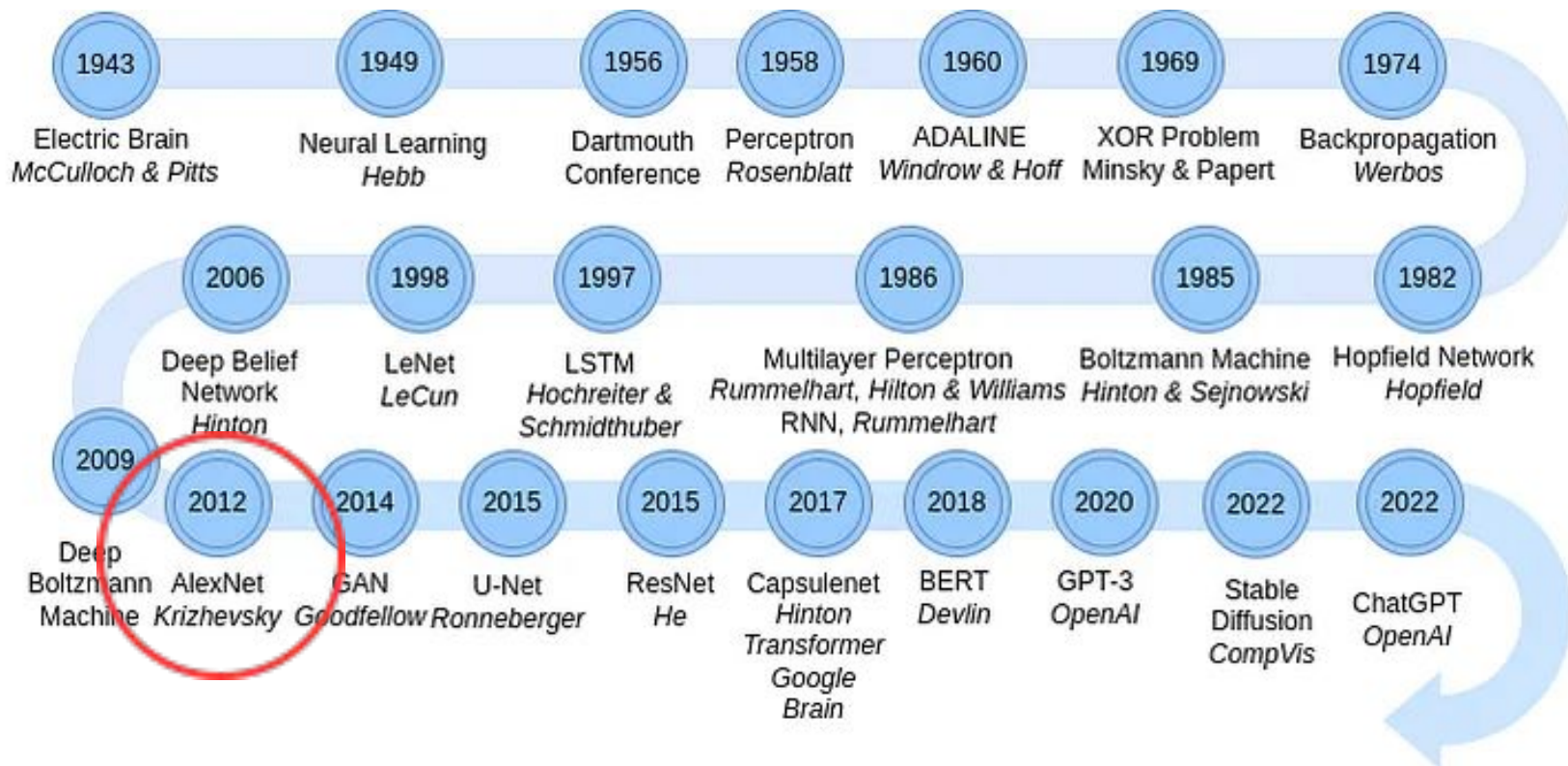


# LeNet-5 (1989-1998)

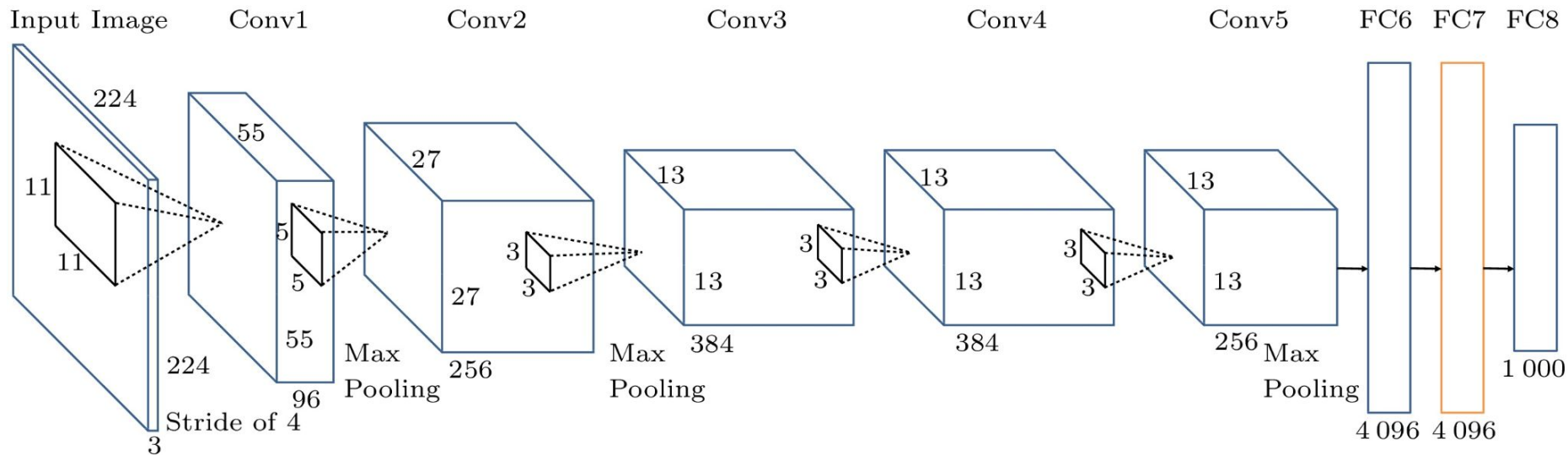


<https://medium.com/appyhigh-technology-blog/convolutional-neural-networks-a-brief-history-of-their-evolution-ee3405568597>

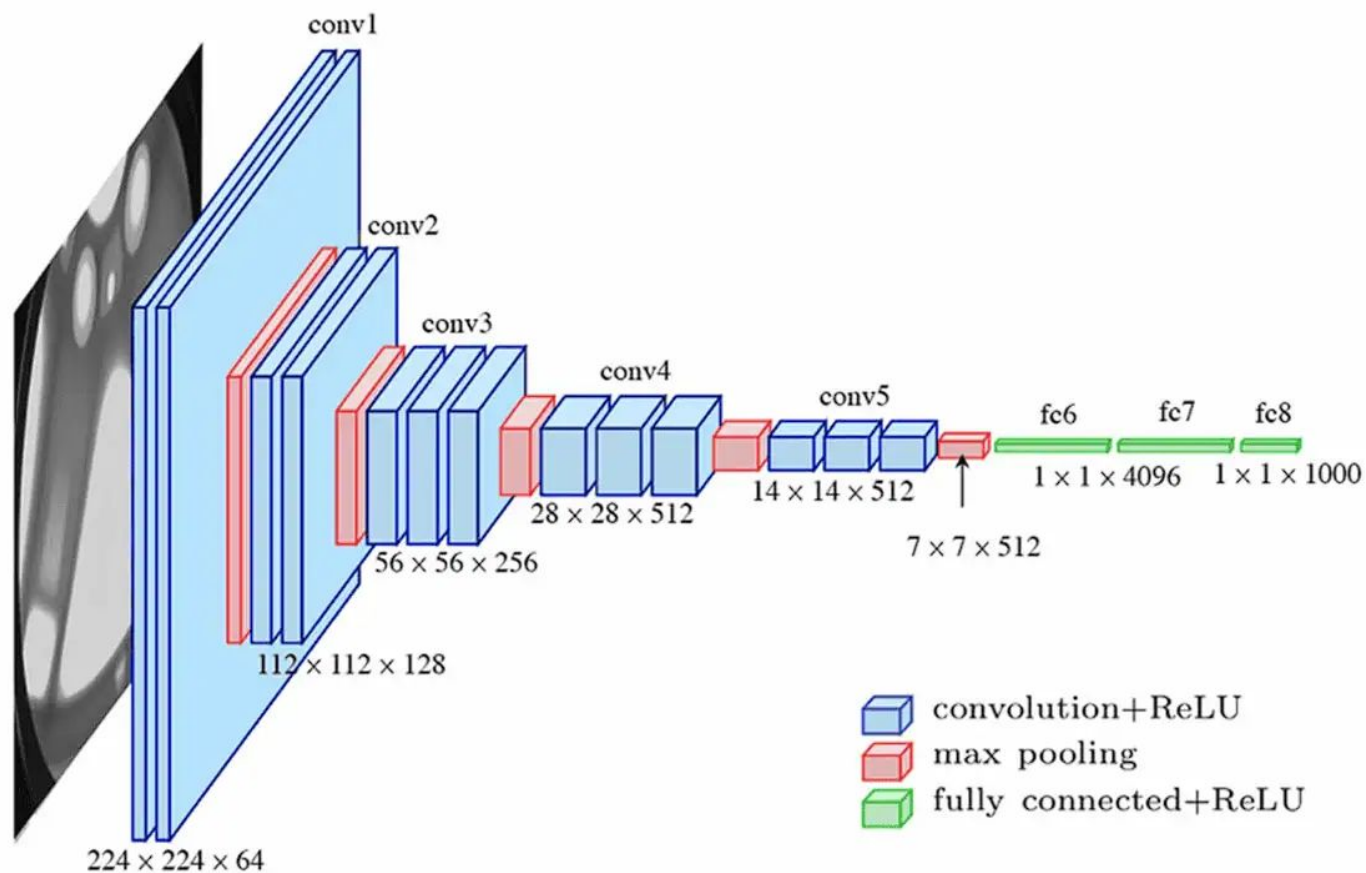
# AlexNet (2012)



# AlexNet (2012)



# VGGNet (2014)



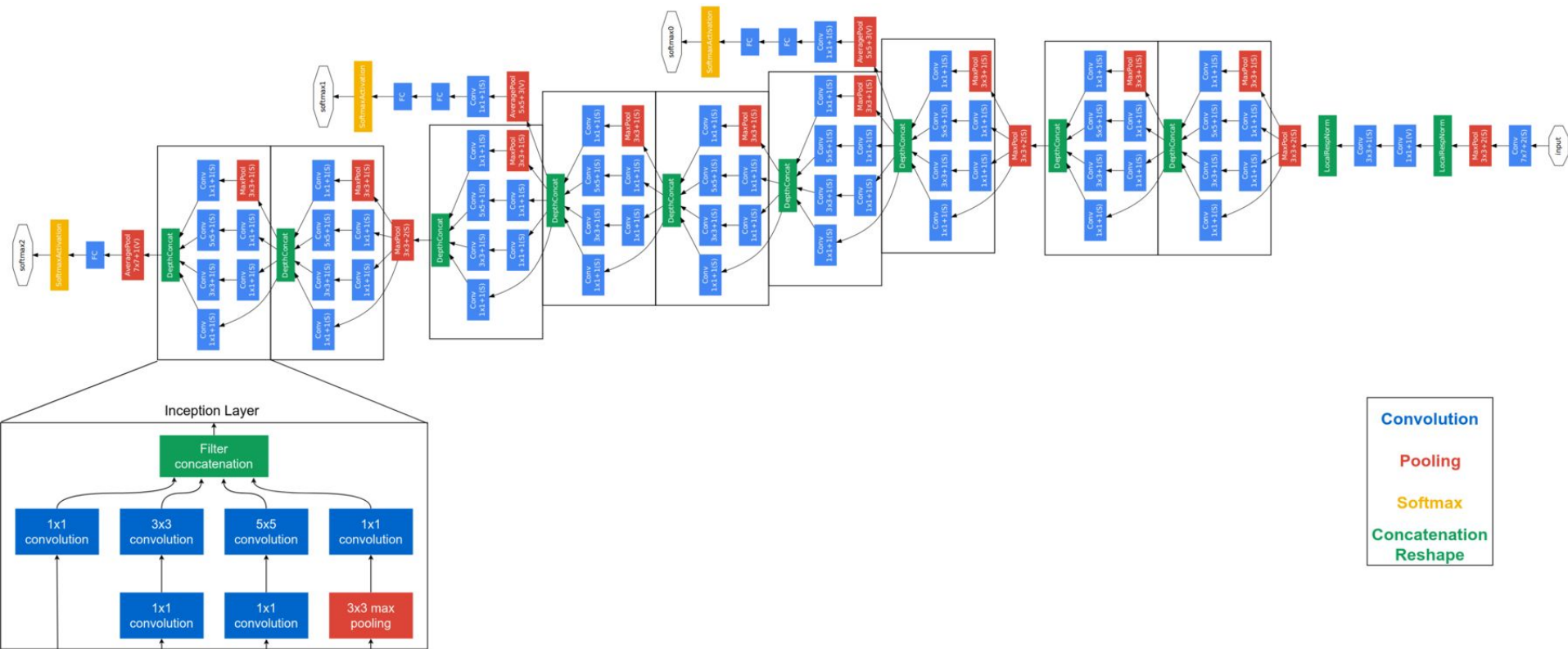


A close-up shot of Leonardo DiCaprio in a dark suit, white shirt, and patterned tie. He is looking towards the right with a serious expression. Another man's profile is visible on the right side of the frame. The background is dark and out of focus.

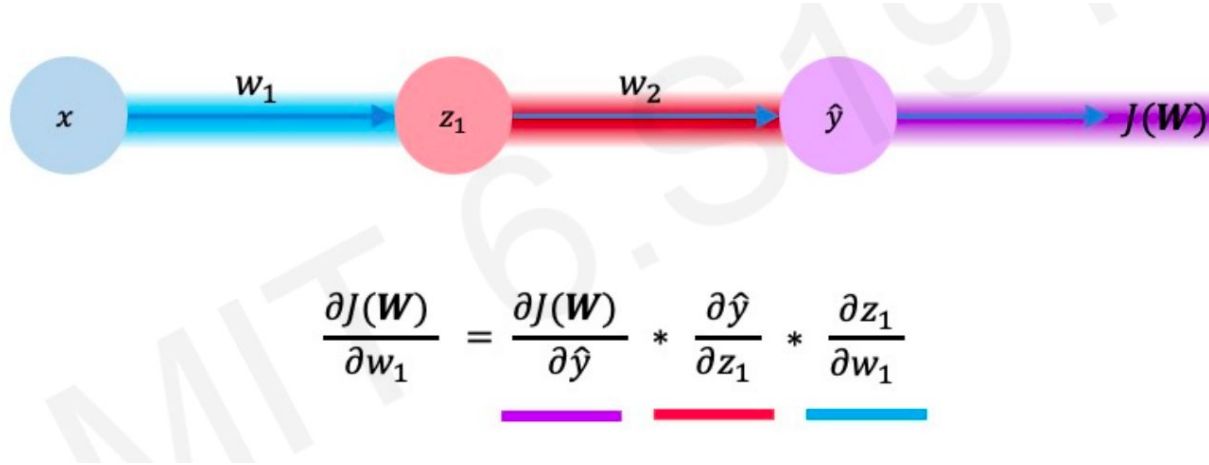
**WE NEED TO GO**

**DEEPER**

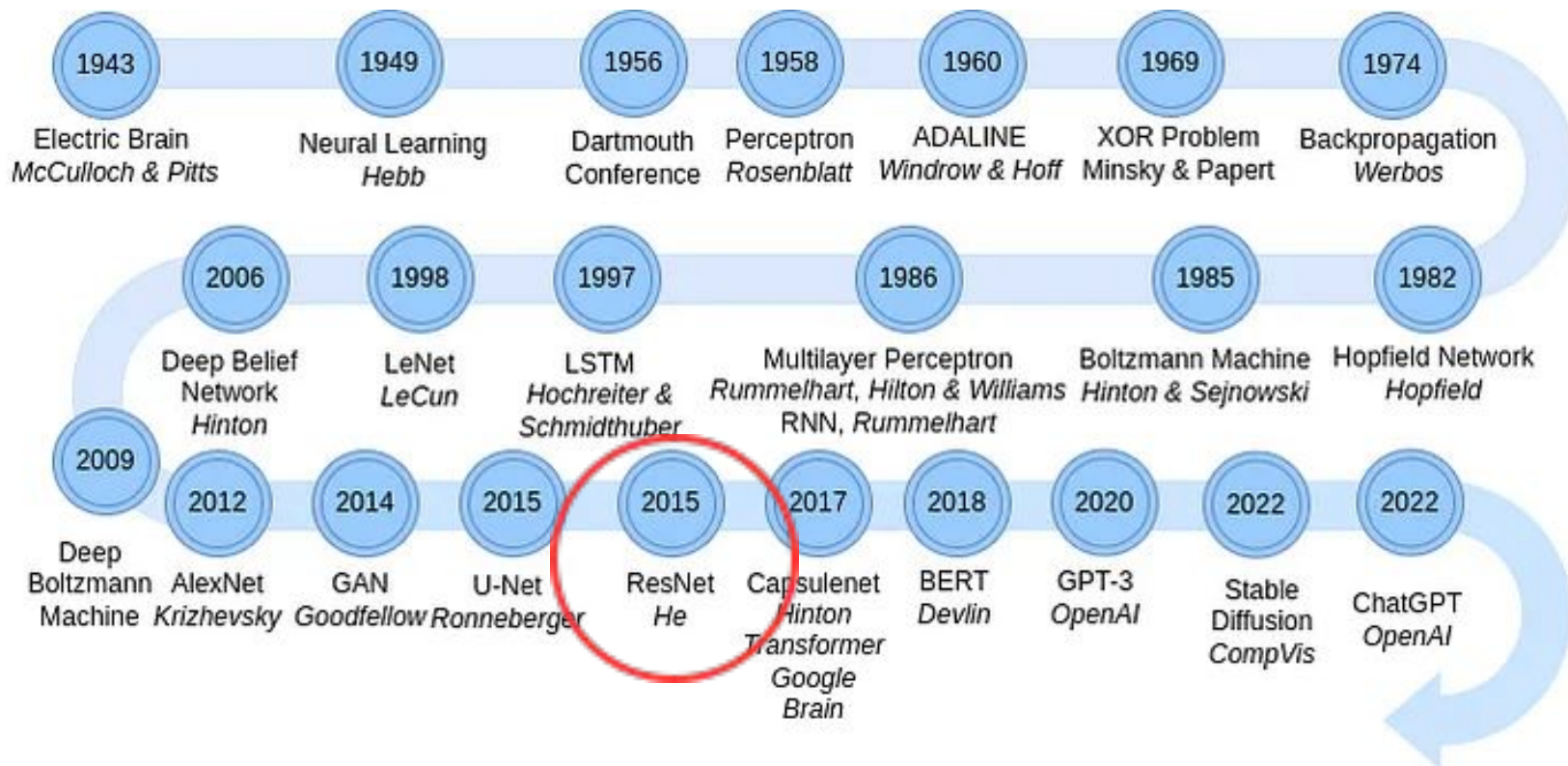
# GoogleNet (2014)



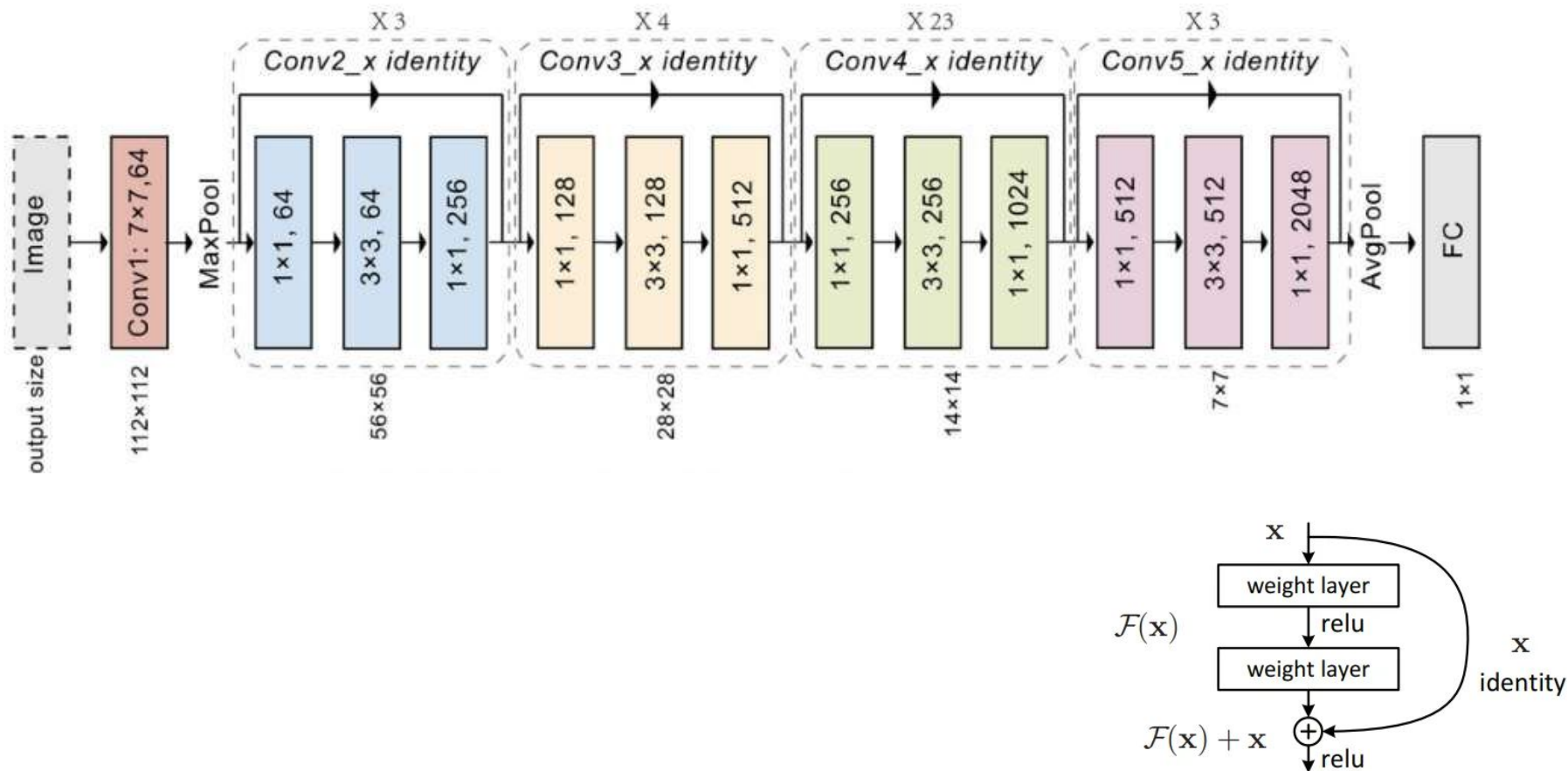
# Vanishing/Exploding Gradients



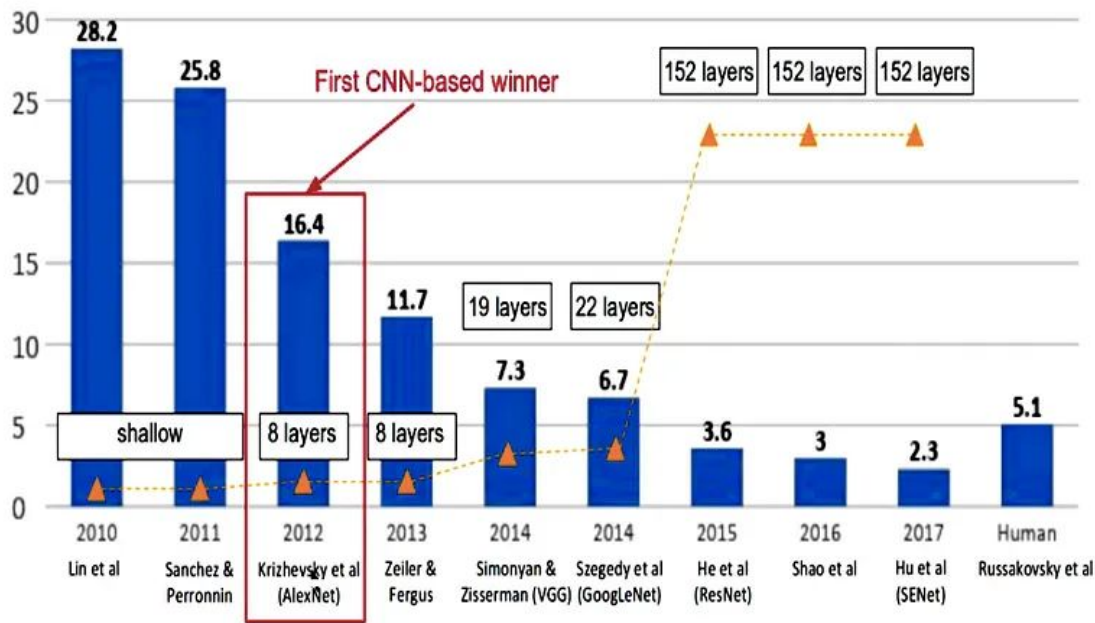
# ResNet (2015)



# ResNet (2015)



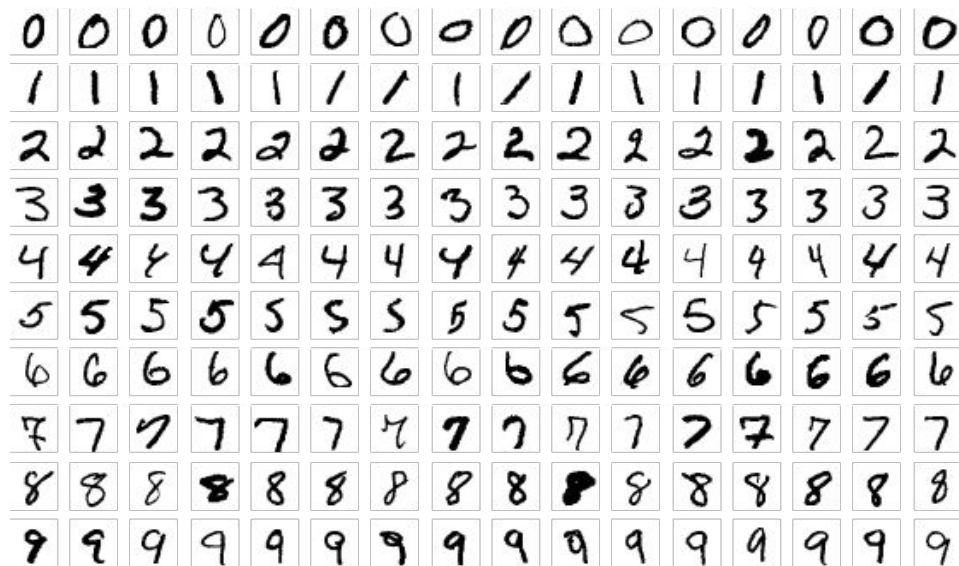
# Winners of the ImageNet classification challenge



<https://medium.com/appyhigh-technology-blog/convolutional-neural-networks-a-brief-history-of-their-evolution-ee3405568597>



# Code Demo - LeNet on MNIST dataset

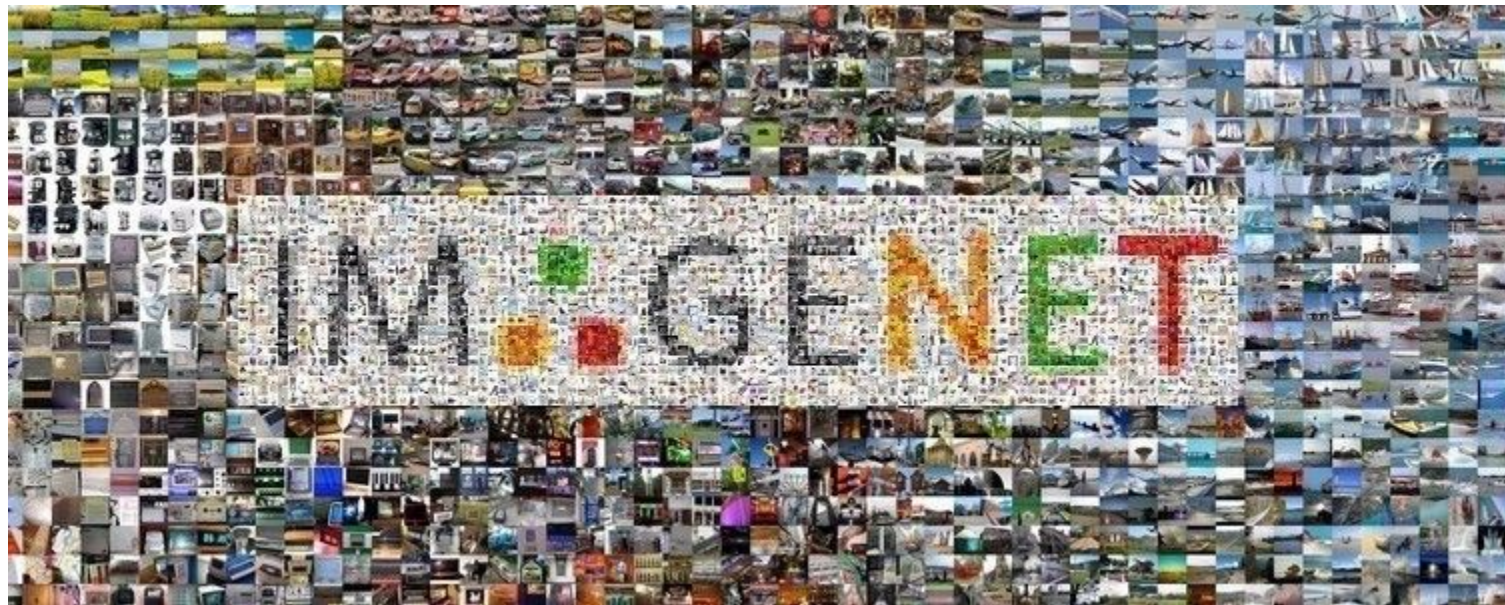


[https://en.wikipedia.org/wiki/MNIST\\_database](https://en.wikipedia.org/wiki/MNIST_database)

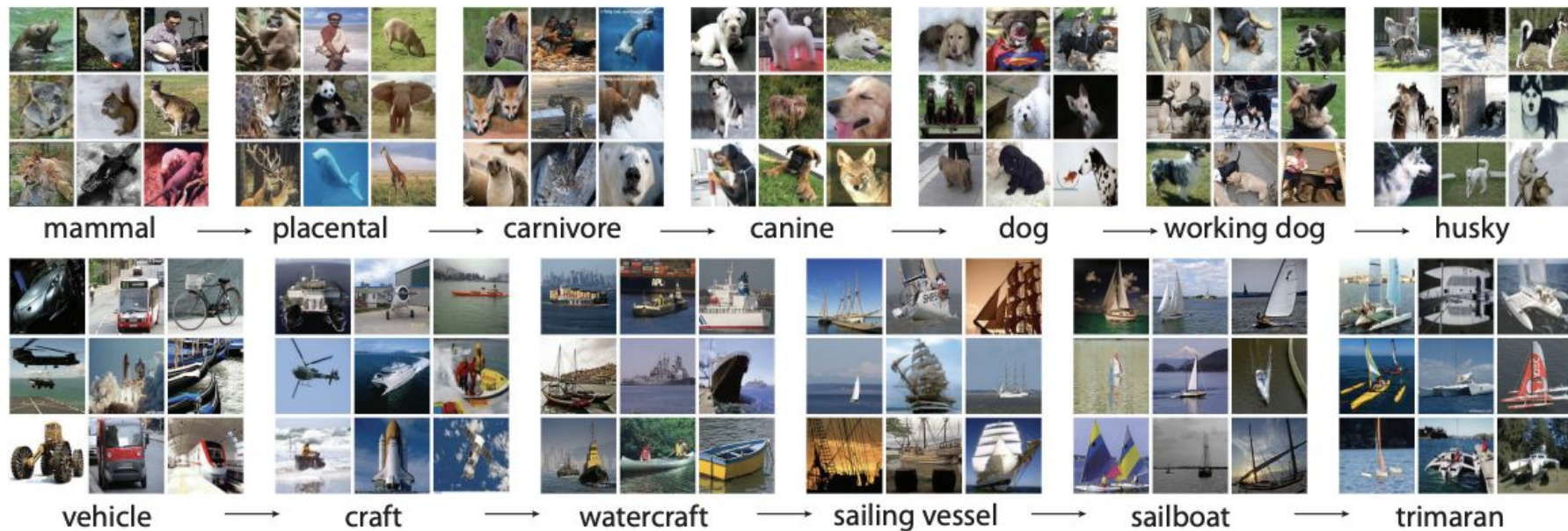


# Transfer Learning and Fine-tuning

# ImageNet

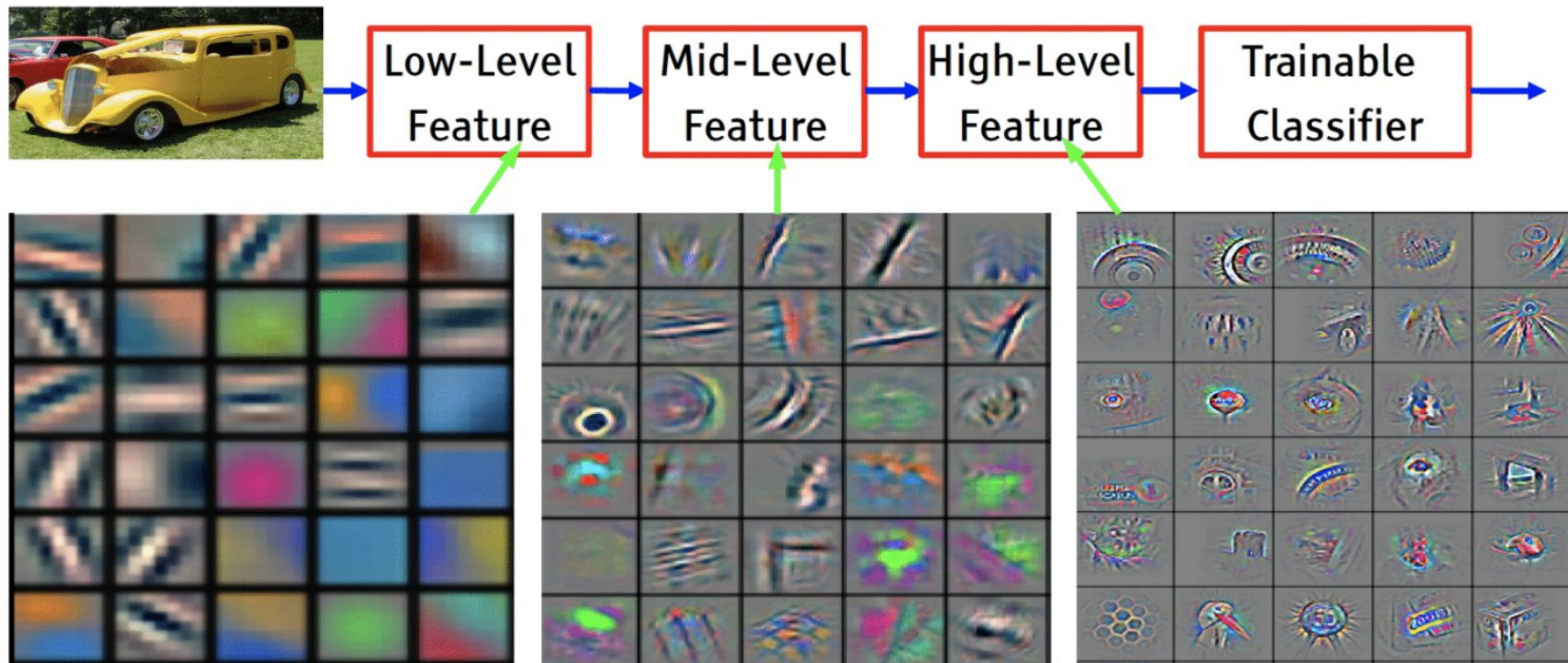


# ImageNet



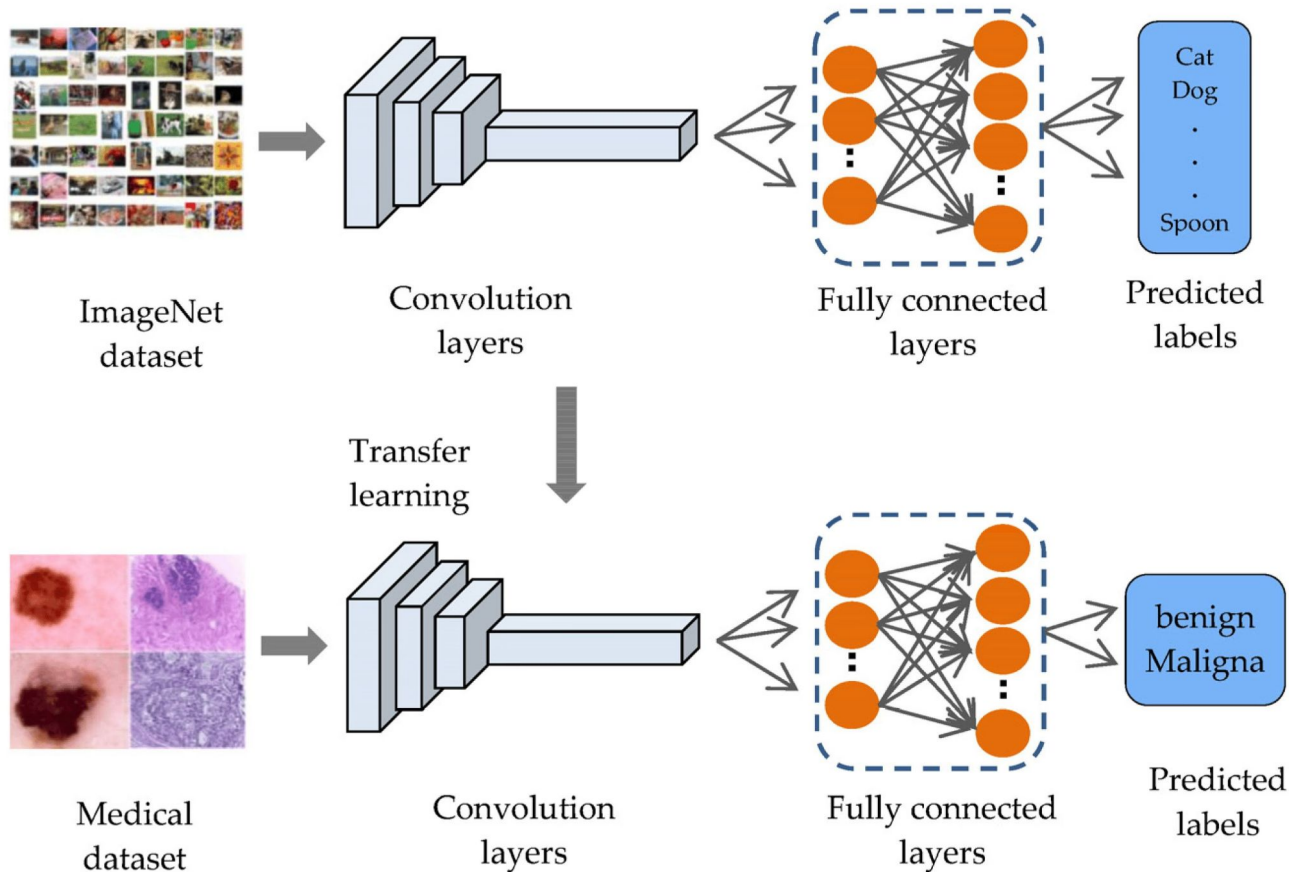


# Feature Hierarchy of Pretrained CNNs



# Transfer Learning

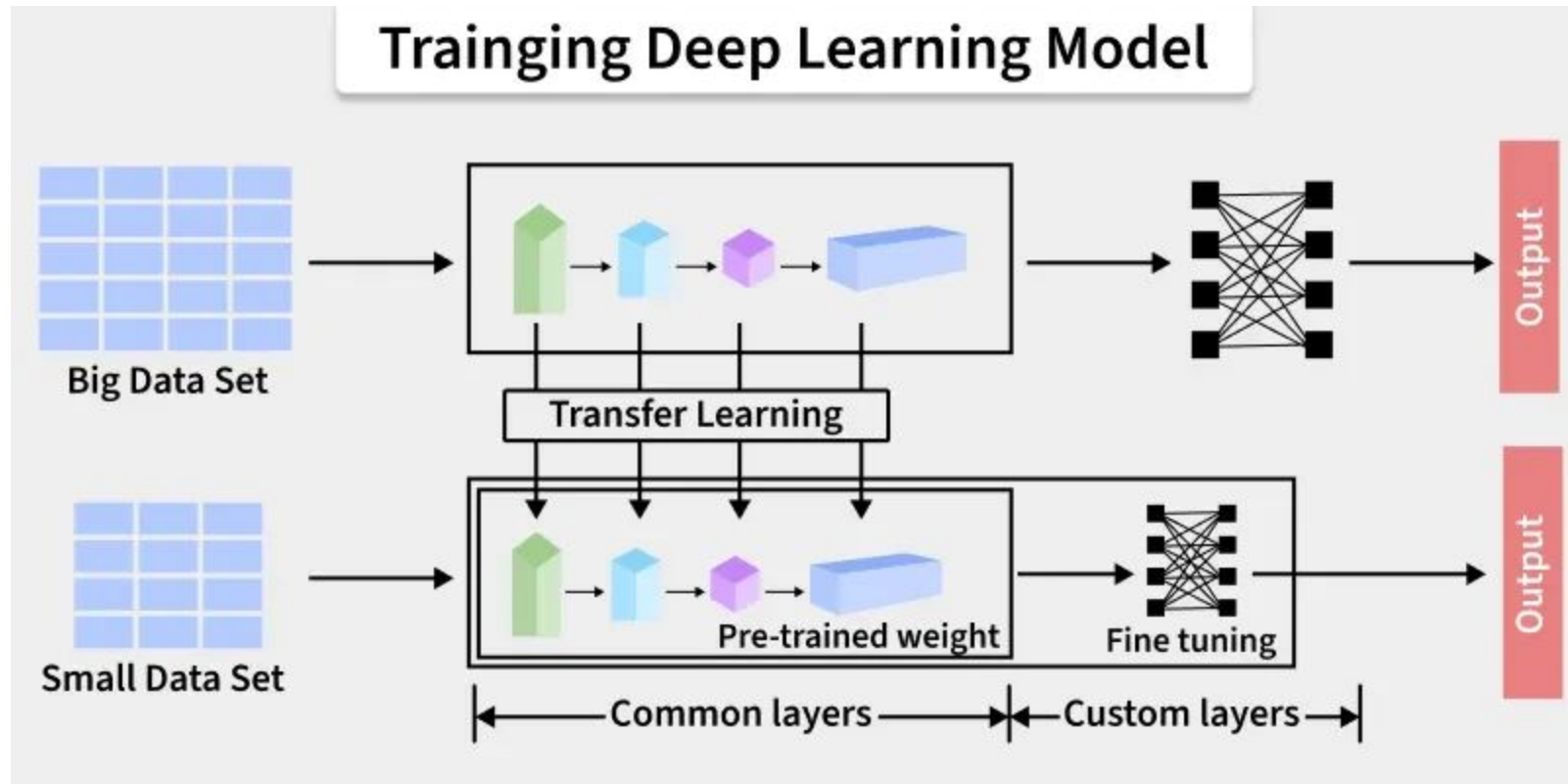
# Transfer Learning



Fine-tuning



# Fine-tuning



# Catastrophic forgetting

- Unfreezing more layers
- Higher learning rates
- Similar/small datasets

# Transfer Learning vs Fine-Tuning

| Feature        | Transfer Learning | Fine-Tuning        |
|----------------|-------------------|--------------------|
| Backbone       | Frozen (Locked)   | Partially Unfrozen |
| Learning Rate  | Normal            | Very Small         |
| Compute Needed | Low               | Moderate           |
| Dataset Size   | Tiny              | More than TL       |